

Thesis Defense Preparation

Potential Questions & Answer Strategies

LBEADS-NET: Algorithm Unrolling for Learned Baseline

Estimation and Denoising in Chromatographic Signals

Saad Zubairi

MS Electrical Engineering, NYU Tandon School of Engineering

Advisor: Professor Ivan Selesnick

April 2026

This document contains **65 anticipated defense questions** organized into eleven categories, each with a suggested answer strategy and references to specific thesis sections, tables, and equations. Section 11 contains the **killer questions**—the hardest, most uncomfortable challenges. A final section catalogues known inconsistencies to fix.

Contents

1 Foundational / “Why This Problem” Questions	3
2 Algorithm Unrolling Theory	5
3 Architecture & Design Choices	7
4 Loss Function Deep Dives	9
5 Experimental Methodology	11
6 Results Interpretation	13
7 Baseline Leakage Deep Dive	15
8 Practical / “So What” Questions	17
9 Writing & Completeness	19
10 Future Work & Big Picture	20
11 Killer Questions — The Ones That Hurt	22
12 Known Inconsistencies & Issues to Fix Before Defense	32
General Defense Advice	33
Glossary of Key Terms	34
Loss Weight Reference (α values)	35
Loss Function Specifications	38
100 Basic Questions & Short Answers	42

1 Foundational / “Why This Problem” Questions

Q1

Why chromatographic baseline correction specifically? Why not apply algorithm unrolling to general signal denoising, where the impact would be broader?

Answer Strategy

Key points:

- Chromatography has a *domain-specific structure* that generic denoisers ignore: the signal decomposes into exactly three additive components—baseline (**b**), peaks (**p**), and noise (ϵ)—with distinct spectral and sparsity priors (Sec. 2.1).
- BEADS already encodes those priors via asymmetric penalty and highpass filtering. Unrolling preserves the interpretable structure while letting each layer adapt.
- Regulatory context matters: pharmaceutical QC (USP <621>) and metabolomics both require *auditable* preprocessing, which favours interpretable, model-based architectures over black-box nets.
- General denoising is well-served by DnCNN / U-Net variants; baseline estimation is an under-explored niche where model-based deep learning is a natural fit.

Thesis references: Sec. 1.1 (motivation), Sec. 2.1 (chromatographic model), Sec. 2.3.3 (BEADS formulation).

Q2

What is wrong with just using arPLS or airPLS? They are simpler, they do not need training data, and they are already widely adopted in chemometrics. Convince me that the added complexity of unrolling is worth it.

Answer Strategy

Key points:

- arPLS/airPLS are iteratively-reweighted penalized least-squares methods. They require manual selection of the smoothness parameter λ and the asymmetry weight p , which are signal-dependent (Sec. 2.3.1–2.3.2).
- They perform *only* baseline estimation—they do not simultaneously denoise. BEADS and LBEADS-NET jointly estimate baseline and denoise.
- The core contribution is not “better numbers than arPLS” but *demonstrating the unrolling paradigm* in this domain and exposing the fundamental ℓ_1 leakage limitation.
- Concede honestly: on well-tuned, non-dense-peak data, arPLS will often suffice. LBEADS-NET is interesting as a research vehicle for understanding leakage.

Thesis references: Table 2.1 (baseline methods comparison), Sec. 2.3.1–2.3.2 (arPLS, airPLS).

Q3

Why does baseline drift matter quantitatively? What error magnitudes are we talking about in real instruments, and how does that translate to, say, drug concentration accuracy?

Answer Strategy

Key points:

- Column bleed, solvent gradients, and detector drift produce baseline offsets of 1–10% of the tallest peak height in HPLC (Sec. 2.1.2).
- In pharmaceutical QC, area-percent quantitation is the norm. A 2% baseline offset can shift a peak area by 5–15%, which exceeds the typical $\pm 2\%$ USP acceptance criterion.
- Metabolomics (hundreds of analytes per run) amplifies the problem: even small per-peak errors compound across the profile.
- Acknowledge: we do not empirically quantify this in the thesis because we work with synthetic data. Cite the chromatography literature for representative magnitudes.

Thesis references: Sec. 2.1.2 (baseline sources), Sec. 1.1 (quantitative motivation).

Q4

Who actually uses this in practice—pharmaceutical QC, metabolomics, environmental labs? Have you talked to anyone who processes real chromatograms and asked whether they need this?

Answer Strategy

Key points:

- Be honest: this is a methods thesis, not a deployment study. No practitioner interviews were conducted.
- Frame the contribution as *foundational analysis*: we unrolled BEADS, exposed the ℓ_1 leakage limit, and documented the failure modes. That knowledge benefits future applied work.
- Note that Professor Selesnick co-authored the original BEADS algorithm, so this work sits in a signal-processing research lineage rather than a product-development one.
- Mention potential users: Agilent OpenLab, Waters Empower, Shimadzu LabSolutions all rely on classical baseline algorithms that could benefit from learned parameter tuning.

Thesis references: Sec. 1.1, Sec. 6 (future work).

Q5

Why not just use a black-box neural network—a U-Net or a 1-D ResNet? They clearly generalise better and do not suffer from ℓ_1 leakage. What does unrolling buy you over a purely learned architecture?

Answer Strategy

Key points:

- **Parameter efficiency:** LBEADS-NET has **100 learnable scalars** (5×20 layers). A comparable 1-D U-Net would have 10^5 – 10^6 parameters, requiring far more training data than our 200 synthetic signals (Sec. 3.2, Sec. 4.1).
- **Interpretability:** every learned parameter has a physical meaning— λ_0 controls fidelity, λ_1/λ_2 control sparsity, r controls penalty shape, η is the step size. This matters for regulatory contexts.
- **Inductive bias:** the unrolled architecture *guarantees* the three-component decomposition structure. A black-box net must learn this from data alone.
- **Honest concession:** black-box nets would likely achieve lower reconstruction error given enough data. The trade-off is interpretability and data efficiency versus raw performance.

Thesis references: Sec. 2.4 (algorithm unrolling theory), Sec. 3.2 (architecture), Table 3.2 (parameter count).

2 Algorithm Unrolling Theory

Q6

What convergence guarantees does LBEADS-NET inherit from the classical BEADS algorithm? Can you prove that the learned iterates converge to anything meaningful?

Answer Strategy

Key points (this is a trap question—acknowledge it):

- Classical BEADS is derived via majorisation-minimisation (MM) and converges to the global optimum of its convex cost function (Sec. 2.3.3, Eq. 2.24).
- **Untied parameters break MM convergence.** Each layer minimises a *different* surrogate, so the sequence of iterates does not descend a single objective.
- This is a known trade-off in all algorithm unrolling: we sacrifice convergence guarantees for task-specific performance via end-to-end training (Sec. 2.4.3).
- Cite Chen & Pock (2017) and Monga et al. (2021): unrolled networks should be understood as *feed-forward architectures with structured layers*, not as truncated optimisers.
- The network is still well-posed in that training converges and test loss is stable—but “convergence of the iterates” in the MM sense does not apply.

Thesis references: Sec. 2.4.3 (unrolling trade-offs), Sec. 3.3 (ISTA reformulation).

Q7

Why ISTA over FISTA? You lose the $O(1/k^2)$ momentum acceleration for free. Was this a deliberate choice or just simpler to implement?

Answer Strategy

Key points:

- The primary reason was **avoiding gradient vanishing through $K \times K_{\text{inner}}$ depth**. Classical BEADS uses a conjugate-gradient inner loop; unrolling both outer MM and inner CG iterations creates excessive depth (Sec. 3.3).
- ISTA replaces the inner CG loop entirely: each unrolled layer is a single proximal-gradient step, giving depth K instead of $K \times K_{\text{inner}}$.
- FISTA’s momentum term introduces a non-monotone trajectory that complicates backpropagation through the unrolled graph (the Nesterov correction depends on the *previous two* iterates, increasing memory footprint).
- In the unrolled setting with untied parameters, the step-size $\eta^{(k)}$ is learned per layer and effectively subsumes much of the acceleration benefit.
- Concede: FISTA layers are a natural extension for future work.

Thesis references: Sec. 3.3 (ISTA reformulation), Sec. 2.4.2 (LISTA and ISTA-based unrolling).

Q8

How does your approach differ from LISTA fundamentally? LISTA learns entire weight matrices, whereas you learn only scalars. Is LBEADS-NET actually “unrolling” or just “parameterised iteration”?

Answer Strategy

Key points:

- LISTA (Gregor & LeCun, 2010) unrolls ISTA for sparse coding: it learns the dictionary $\mathbf{W}$ and the Gram matrix $\mathbf{S}$ as free matrices, so each layer is a full affine transform (Sec. 2.4.2).
- LBEADS-NET retains the *fixed structural operators* ($\mathbf{H}$ highpass filter, $\mathbf{D}$ difference matrices) and only unties the *scalar regularisation weights* across layers—5 scalars per layer.
- This is a design choice for interpretability and parameter efficiency, not a limitation. The structural operators encode domain knowledge (filter design, finite differences) that should *not* be made freely learnable with only 200 training signals.
- It is still correctly termed “unrolling”: each layer corresponds to one iteration of a known algorithm with layer-specific parameters. The Monga et al. (2021) taxonomy calls this “algorithm unrolling with untied hyperparameters.”

Thesis references: Sec. 2.4.2 (LISTA), Sec. 3.2 (LBEADS-NET architecture), Table 3.2.

Q9

What happens if you tie all parameters across layers—that is, force $\lambda_0^{(k)} = \lambda_0$ for all k ? Does it reduce to classical BEADS? Have you tried it?

Answer Strategy

Key points:

- With tied parameters, each layer applies the *same* proximal-gradient step, so the unrolled network becomes K iterations of ISTA applied to the BEADS cost function with *learned* but *shared* regularisation weights.
- It does **not** exactly reduce to classical BEADS because (a) BEADS uses MM with CG inner solve, not ISTA, and (b) the $K = 20$ iteration budget may not suffice for convergence of the tied-parameter ISTA.
- The tied-parameter variant is an important ablation that isolates the benefit of *per-layer specialisation*.
- Concede: this ablation is listed in Table 5.3 but has not been run yet. It is planned before the final submission.
- The emergent parameter specialisation (Sec. 5.2.3) suggests that untying is beneficial: early layers learn coarse separation, later layers learn aggressive refinement.

Thesis references: Sec. 3.2 (architecture), Sec. 5.2.3 (parameter analysis), Table 5.3 (ablation, currently empty).

Q10

The BEADS cost function is convex. Does unrolling preserve convexity of the end-to-end mapping? If not, what are the optimisation implications for training?

Answer Strategy

Key points:

- **No.** The end-to-end mapping $\mathbf{y} \mapsto (\hat{\mathbf{b}}, \hat{\mathbf{p}})$ is a composition of 20 layers, each involving soft-thresholding (non-linear) and matrix multiplications parameterised by learned scalars. The composition is *not* convex in the parameters $\{\lambda_0^{(k)}, \dots, \eta^{(k)}\}$.
- The *training loss landscape* is therefore non-convex, which is standard for neural networks.
- However, each *individual layer* is a proximal operator applied to a convex penalty, which provides local regularity (Lipschitz continuity of the proximal map).
- Practically, the 3-stage curriculum (Sec. 3.8) and progressive loss introduction help navigate the non-convex landscape by providing a good initialisation path.

Thesis references: Sec. 2.3.3 (BEADS convexity), Sec. 3.3 (ISTA layers), Sec. 3.8 (curriculum training).

3 Architecture & Design Choices

Q11

Why $K=20$ layers? Your ablation table (Table 5.3) is completely empty. How do you justify this choice without any empirical evidence?

Answer Strategy

Key points:

- Honest answer: $K=20$ was chosen heuristically based on (a) the classical BEADS default of 20–30 outer iterations and (b) GPU memory constraints with $N=4096$ and dense matrices.
- The parameter analysis in Sec. 5.2.3 was conducted with $K=6$ and $N=1024$, which is not directly comparable.
- Acknowledge that the empty ablation table is a gap. State that $K \in \{6, 10, 15, 20, 30\}$ should be swept.
- Qualitative argument: BEADS typically converges in ~ 15 –20 iterations, so the unrolled variant should need *at most* that many layers (and likely fewer, since parameters are untied).
- Cite the LISTA literature: Borgerding et al. (2017) show that unrolled ISTA with 15–20 layers matches 100+ iterations of classical ISTA.

Thesis references: Table 5.3 (empty ablation), Sec. 5.2.3 (6-layer analysis), Sec. 3.2.

Q12

Your implementation uses dense $N \times N$ matrices with $O(N^2)$ memory. For $N=4096$ in float64, each matrix is 128 MB. With 20 layers, you are storing gigabytes of intermediate buffers. Why not exploit the banded structure of $\mathbf{H}$ and $\mathbf{D}$?

Answer Strategy

Key points:

- You are correct—this is the single largest engineering limitation of the current implementation.
- The highpass filter $\mathbf{H}$ is a banded Toeplitz matrix and $\mathbf{D}$ (first/second difference) is banded with bandwidth 1–2. Products $\mathbf{H}^T \mathbf{H}$ and $\mathbf{D}^T \mathbf{D}$ are also banded.
- The reason for dense matrices: the filter $\mathbf{H}$ is constructed via `scipy.signal` and converted to a dense NumPy array before wrapping in a PyTorch tensor. Implementing banded-matrix operations natively in PyTorch would require custom CUDA kernels or use of `torch.sparse`.
- This is an engineering priority for future work. Banded storage would reduce memory from $O(N^2)$ to $O(N \cdot b)$ where b is the bandwidth ($b \ll N$), enabling $N=65536+$ signals.
- Practically, the current implementation fits on a single GPU for $N=4096$ only because of small batch sizes.

Thesis references: Sec. 3.4 (filter construction), Sec. 6 (future work).

Q13

You say f_c (the highpass cutoff frequency) is not learnable because of the “SciPy–PyTorch autograd boundary.” But why not implement the entire filter construction in PyTorch? The Butterworth poles-and-zeros are closed-form.

Answer Strategy

Key points:

- The current implementation calls `scipy.signal.butter()` and `scipy.signal.tf2zpk()`, which are not differentiable through PyTorch’s autograd (Sec. 3.4).
- In principle, the bilinear-transform design of a Butterworth filter *can* be written as closed-form operations on f_c (pole placement on the unit circle), and these operations are differentiable.
- The difficulty is constructing the *banded matrix* $\mathbf{H}(f_c)$ from those poles in a way that is compatible with efficient batched backpropagation.
- A more practical route: parameterise the filter as a learnable FIR filter of fixed order and initialise it from the Butterworth design. This avoids the IIR stability issues.
- This is listed as future work (Sec. 6) and would add one more learnable parameter per layer (101 total scalars).

Thesis references: Sec. 3.4 (filter construction), Eq. 3.5–3.6, Sec. 6 (future work).

Q14

What is the effective receptive field of LBEADS-NET? Each layer operates on the full signal via dense matrix multiplication, so is the receptive field trivially the entire signal?

Answer Strategy

Key points:

- Yes—formally, every output sample depends on every input sample after a single layer, because $\mathbf{H}$ is an IIR filter realised as a dense matrix (Sec. 3.4).
- However, the *effective* receptive field (in the Luo et al. 2016 sense) is narrower because the energy of $\mathbf{H}$ decays exponentially away from the diagonal.
- The practical receptive field is governed by the filter order and $f_c=0.006$: the -3 dB point at 0.6% of Nyquist corresponds to a spatial scale of ~ 167 samples.
- For baseline estimation, a large receptive field is *desirable*: the baseline is a slowly-varying component that depends on global signal context.
- For peak estimation, local structure matters more, which is why the sparsity penalties (ℓ_1 , TV) act element-wise or on adjacent differences.

Thesis references: Sec. 3.4 (filter design), Sec. 2.3.3 (BEADS formulation).

Q15

Why float64 instead of float32? Is this a fundamental numerical requirement of the algorithm, or is it just an engineering choice that doubles your memory usage?

Answer Strategy

Key points:

- It is partially fundamental: the highpass filter $\mathbf{H}$ at $f_c=0.006$ has eigenvalues spanning several orders of magnitude. In float32, the matrix $\mathbf{H}^T \mathbf{H} + \lambda_0 \mathbf{I}$ can become poorly conditioned and produce NaN gradients during backpropagation.
- Empirically, early experiments in float32 showed training instability (gradient overflow in layers 15–20).
- However, with banded solvers or mixed-precision strategies (float64 for the filter, float32 for the rest), it should be possible to reduce memory. This was not explored.
- The ISTA reformulation partially mitigates the conditioning issue (no matrix inverse required), so float32 may be viable with careful step-size initialisation.

Thesis references: Sec. 3.4 (numerical considerations), Sec. 4.2 (training configuration).

Q16

You mention using *softplus* for non-negativity constraints in v7 but not in v5. *Softplus* versus *ReLU*—what is the difference in this context, and why does it matter?

Answer Strategy

Key points:

- The regularisation parameters $\lambda_0, \lambda_1, \lambda_2, r, \eta$ must be strictly positive. A reparameterisation is needed: the network stores unconstrained $\theta^{(k)}$ and maps to $\lambda^{(k)} = g(\theta^{(k)})$.
- **ReLU:** $g(\theta) = \max(0, \theta)$. Has zero gradient for $\theta < 0$ (“dead parameter” problem) and is non-differentiable at 0.
- **Softplus:** $g(\theta) = \log(1 + e^\theta)$. Smooth, strictly positive, non-zero gradient everywhere.
- In v7, softplus was adopted after observing that some parameters collapsed to zero under ReLU and could not recover (gradient was exactly zero).
- Softplus provides a mild bias toward small positive values rather than exact zeros, which is physically appropriate—the regularisation weights should not be zero.

Thesis references: Sec. 3.6 (parameter constraints), Sec. 3.7.1 (v7 modifications).

4 Loss Function Deep Dives

Q17

You have 12 loss terms with 12 hyperparameters (the weights). Classical BEADS has 6 parameters. Haven’t you just replaced one tuning problem with a larger one? How is this progress?

Answer Strategy

Key points:

- Fair criticism—acknowledge it directly.
- The critical difference: BEADS’s 6 parameters must be tuned *per signal* (or per instrument/method), whereas the 12 loss weights are set *once during training* and the network then adapts its 100 per-layer parameters to each new signal at inference time.
- The loss weights are meta-parameters that define the *training objective*, not the inference-time model. At test time, LBEADS-NET has zero user-tunable parameters.
- Concede: the proliferation of loss terms (especially the 12-term v7 variant) is itself a finding—Experiment 1.5 shows that the full 12-term loss *hurts* performance due to competing gradients (Sec. 5.2).
- The thesis’s *negative result*—that more loss terms do not help—is itself a contribution.

Thesis references: Table 3.3 (loss terms), Sec. 5.2.1 (Exp. 1.5 results), Sec. 3.7.

Q18

The orthogonality loss $\mathcal{L}_{orth}$ uses `.detach()`’d ground-truth-derived weights. Isn’t this a form of label leakage during training? You are giving the network access to the answer.

Answer Strategy

Key points:

- Clarify what happens: the ground-truth peak signal is used to construct a *weighting mask* that emphasises peak regions, and this mask is detached (no gradient flows through it). The loss penalises $\langle \mathbf{b}, \mathbf{w} \rangle$ where $\mathbf{w}$ is the detached mask.
- This is **not** label leakage in the test-time sense: the mask is used only during training (supervised learning always uses ground truth). At inference, no ground truth is available or used.
- The `.detach()` prevents the gradient from flowing back through the mask construction, which would create a circular dependency.
- Analogy: it is similar to importance sampling in RL, where the sampling distribution depends on the target but gradients are blocked through the weights.
- However, this does make the loss non-differentiable with respect to the mask shape, so the network cannot learn to “game” the mask.

Thesis references: Sec. 3.7 (loss function definition), Eq. 3.18 (orthogonality loss).

Q19

Your *asymmetric baseline loss* uses $\alpha=0.9$ (a 9:1 ratio penalising over-estimation versus under-estimation). How sensitive is the model to this choice? Did you try $\alpha=0.7$ or $\alpha=0.95$?

Answer Strategy

Key points:

- The intuition for $\alpha=0.9$: baseline over-estimation absorbs peak energy (leakage), which is harder to correct downstream than a small under-estimation. So we penalise over-estimation $9\times$ more heavily.
- Concede: a sensitivity sweep over α was not performed. This is a gap.
- Argue that the exact value matters less than the *qualitative asymmetry*: any $\alpha > 0.5$ biases the baseline downward, and the effect saturates.
- Very high α (e.g., 0.99) would force the baseline to zero everywhere, destroying the baseline estimate. Very low α (e.g., 0.6) provides weak anti-leakage pressure.
- A proper sweep (Table 5.4) is listed but not yet run.

Thesis references: Sec. 3.7 (asymmetric loss), Table 5.4 (sensitivity ablation, currently empty).

Q20

The *envelope constraint* uses a *soft-minimum* with temperature $\tau=0.1$ and a window size of 51. How sensitive are the results to these choices? Why 51?

Answer Strategy

Key points:

- Window size 51 corresponds to roughly the half-width of a typical synthetic peak ($\sigma \approx 20\text{--}30$ samples), so the envelope captures the local minimum in the valley between adjacent peaks.
- Temperature $\tau=0.1$ makes the soft-min approximate a hard min: $\text{softmin}_\tau(\mathbf{x}) \rightarrow \min(\mathbf{x})$ as $\tau \rightarrow 0$. At $\tau=0.1$, the approximation is tight while remaining differentiable.
- Sensitivity: smaller windows (~ 21) would miss wide peaks; larger windows (~ 101) would create a baseline that cannot follow moderate-frequency drift.
- This was not swept systematically. The values were set once based on the known peak widths in the synthetic data.
- On real data with unknown peak widths, these would need adaptive selection—a limitation.

Thesis references: Sec. 3.7 (envelope constraint definition).

Q21

You describe the ℓ_1 and TV penalties as both the cause of baseline leakage and necessary for sparsity in non-peak regions. This seems like a fundamental contradiction. How do you resolve it?

Answer Strategy

Key points:

- This is the **central tension of the thesis**—do not try to resolve it cleanly, because it *is not resolved*.
- ℓ_1 promotes sparsity in $\mathbf{p}$, which is correct in regions with no peaks (sparse support). But in dense-peak regions, ℓ_1 biases peak amplitudes toward zero (*soft-thresholding shrinkage bias*, well-known in compressed sensing).
- The energy removed from peaks has to go somewhere in the $\mathbf{y} = \mathbf{b} + \mathbf{p} + \boldsymbol{\varepsilon}$ decomposition—it leaks into $\hat{\mathbf{b}}$.
- The 12-term loss attempts to *mitigate* this with anti-leakage terms, but BLI plateaus at ~ 0.59 , confirming that loss engineering cannot overcome the architectural bias.
- **The honest conclusion:** ℓ_1 -based unrolling is fundamentally limited for dense-peak signals. Future work should explore group sparsity, ℓ_0 -like penalties, or hybrid architectures.

Thesis references: Sec. 3.7 (loss design rationale), Sec. 5.2.2 (BLI plateau analysis), Sec. 5.5 (leakage discussion).

Q22

You use Huber loss with $\delta=1.0$ for reconstruction in the main experiments but mention $\delta=0.1$ elsewhere. Which is it, and why Huber over MSE in the first place?

Answer Strategy

Key points:

- Huber loss is piecewise: quadratic for $|e| \leq \delta$, linear for $|e| > \delta$. It down-weights large residuals, providing robustness to outlier peaks.
- $\delta=1.0$: the data is normalised to $[0, 1]$, so $\delta=1.0$ means *almost all residuals fall in the quadratic regime*— $\text{Huber}_{1.0} \approx \text{MSE}$ for normalised data.
- $\delta=0.1$: makes the linear regime much larger, acting more like MAE. This was used in the curriculum’s Stage B (Sec. 3.8) to reduce sensitivity to large leakage residuals during the early anti-leakage phase.
- Acknowledge that the δ value is not consistently documented and should be clarified.
- MSE was not used because early experiments showed that squared large residuals from leakage regions dominated the gradient, destabilising training.

Thesis references: Sec. 3.7 (loss terms), Sec. 3.8 (curriculum training), Table 4.2 (training config).

5 Experimental Methodology

Q23

200 signals total, 40 for testing. How can you draw any statistically meaningful conclusions from 40 test signals? What is your confidence interval on, say, peak correlation?

Answer Strategy

Key points:

- Concede: 40 test signals is small. No confidence intervals or statistical tests are reported, which is a limitation.
- Mitigating factors: each signal contains 8–25 peaks, so the effective sample size for *per-peak* metrics is ~ 600 . Per-signal metrics are averaged over ~ 40 diverse signals.
- The synthetic data generation is controlled, so there is no domain shift between train and test—only random variation in peak parameters.
- The small dataset is *by design*: it tests whether unrolling’s inductive bias can compensate for data scarcity. A 100k-signal dataset would not test this hypothesis.
- For a proper statistical treatment, report mean $\pm$ std and bootstrap confidence intervals. This should be added before the final submission.

Thesis references: Sec. 4.1 (data generation), Sec. 5.1 (evaluation metrics), Table 5.1.

Q24

Your comparison with classical BEADS uses parameters designed for unnormalised data, applied to normalised data. Isn’t this a deliberately unfair comparison that makes BEADS look worse than it is?

Answer Strategy

Key points:

- Partially valid criticism. The default BEADS parameters ($\lambda_0=1, \lambda_1=1, \lambda_2=1$) are recommended for unnormalised chromatograms with peak heights of 10^3 – 10^6 . On $[0, 1]$ -normalised data, these regularisation strengths are relatively too large.
- However: the comparison is intended to show what happens when a user applies BEADS “out of the box” without per-signal tuning. This is the common use case.
- A *fairer* comparison would grid-search BEADS parameters on a validation set. This was not done and should be acknowledged as a limitation.
- The main contribution is not “LBEADS-NET beats BEADS” but rather the analysis of leakage and the unrolling methodology.

Thesis references: Sec. 5.2 (comparison setup), Table 5.1 (results).

Q25

You survey arPLS, airPLS, SNIP, and several deep learning baselines in Chapter 2, but you never compare against any of them experimentally. Why not?

Answer Strategy

Key points:

- Concede: this is a significant omission.
- The reason is scope and time: implementing each baseline with proper hyperparameter tuning on the synthetic dataset would require substantial additional work.
- The thesis’s primary goal is to unroll *BEADS specifically* (given the advisor’s authorship of BEADS), not to conduct a broad benchmark.
- arPLS is the most important missing comparison because it is the most widely used method in practice. This should be added before the defense if possible.
- Deep learning baselines (1-D U-Net) would require architectural design and training, which is a separate study.
- Frame as future work.

Thesis references: Sec. 2.3 (method survey), Table 2.1 (comparison table), Sec. 6 (future work).

Q26

Your synthetic data uses bi-Gaussian peaks (Section 4.1.1), but your background chapter discusses the EMG (Exponentially Modified Gaussian) model in detail (Section 2.1.4). Why the mismatch? Does this limit the relevance of your results?

Answer Strategy

Key points:

- EMG is the standard physical model for chromatographic peaks (mass transfer kinetics cause exponential tailing). Bi-Gaussian is a simpler empirical model that captures asymmetry without the convolution integral.
- Bi-Gaussian was chosen for computational simplicity in synthetic data generation and because it has an analytic closed-form (no numerical convolution needed).
- The two models produce qualitatively similar peak shapes for moderate asymmetry. For heavy tailing, EMG produces longer tails that bi-Gaussian cannot capture.
- This mismatch is not explained in the thesis and should be. Add a brief justification in Sec. 4.1.1.
- The choice does limit relevance: results on bi-Gaussian peaks may not transfer to heavily tailed EMG peaks.

Thesis references: Sec. 2.1.4 (EMG model), Sec. 4.1.1 (bi-Gaussian generation).

Q27

Your synthetic data generation has many design choices: cluster probability 0.4, log-uniform amplitude distributions, specific σ ranges. How sensitive are your results to these choices? Have you tested on a different distribution?

Answer Strategy

Key points:

- These parameters control the “difficulty distribution” of the training set: cluster probability determines how often dense-peak regions occur (which is where leakage is worst).
- No sensitivity analysis on data generation parameters was performed. This is a limitation.
- Higher cluster probability would create more dense-peak signals, likely increasing average BLI. Lower cluster probability would create easier signals with better metrics.
- The log-uniform amplitude distribution ensures coverage of both small and large peaks, which is more realistic than uniform sampling.
- A robust model should be tested on out-of-distribution data (different peak width ranges, different noise levels). This was not done.

Thesis references: Sec. 4.1.1 (data generation parameters), Table 4.1 (generation config).

6 Results Interpretation

Q28

BLI is approximately 0.59 across all configurations—from the simplest 4-term loss to the full 12-term v7 loss. Doesn't this mean your entire loss engineering effort was basically useless for addressing leakage?

Answer Strategy

Key points:

- **Yes, essentially.** This is a key negative result and should be stated directly.
- The BLI plateau at ~ 0.59 across all configurations is strong evidence that leakage is an **architectural** property of ℓ_1 -based unrolling, not a loss-function problem.
- This is a *valuable finding*: it tells future researchers not to invest in loss engineering for this architecture. The fix must be structural (different penalty, different architecture).
- The loss terms *do* help other metrics (reconstruction error, peak shape fidelity), just not leakage.
- Frame this as: “the thesis *discovered* the leakage floor and *proved* that loss engineering cannot break through it.”

Thesis references: Table 5.1 (BLI column), Sec. 5.2.2 (BLI analysis), Sec. 5.5 (leakage discussion).

Q29

Area error is 49–58% across all LBEADS-NET variants. In pharmaceutical QC, 1% accuracy is the requirement. 50% error is catastrophic. How can you call this a contribution?

Answer Strategy

Key points:

- Do not try to defend 50% area error as acceptable—it is not, and any committee member from chromatography would dismiss this immediately.
- Reframe: the contribution is *not* a production-ready tool. It is a **methodological study** that: (a) demonstrates algorithm unrolling for BEADS, (b) identifies and characterises the ℓ_1 leakage limit, (c) documents the failure of loss engineering to fix it.
- The area error is dominated by leakage in dense-peak regions. For isolated peaks, the per-peak area error is much lower (this should be reported separately if data is available).
- The hybrid pipeline (Sec. 3.9) improves area error by applying classical BEADS post-processing, but the fundamental limit remains.
- Honest conclusion: LBEADS-NET in its current form is not suitable for quantitative chromatography.

Thesis references: Table 5.1 (area error), Sec. 5.2 (quantitative results), Sec. 5.5.

Q30

Experiment 1.5 (the full v7 12-term loss) performs worse than Experiment 1.3 (a simpler configuration). Your own results show that simpler is better. Why do you propose the complex version at all?

Answer Strategy

Key points:

- This is a genuinely important finding. The 12-term loss creates **competing gradient directions**: the reconstruction loss pulls parameters one way while the anti-leakage losses pull them another way, resulting in a compromise that is worse on all metrics.
- This is an example of *over-regularisation*—adding too many constraints to a 100-parameter model.
- The simpler Experiment 1.3 succeeds because fewer loss terms give the optimiser a cleaner gradient signal.
- The thesis proposes v7 as a *research contribution* (novel loss formulations) even though v5/Exp. 1.3 is empirically better. The intellectual value is in understanding *why* the complex version fails.
- Recommendation: the thesis should more prominently highlight Exp. 1.3 as the best-performing configuration.

Thesis references: Table 5.1 (Exp. 1.3 vs 1.5), Sec. 5.2.1 (progressive results).

Q31

Peak correlation maxes at 0.793. What does this mean physically? That 79% of the peak shape is recovered? What is an acceptable value in practice?

Answer Strategy

Key points:

- Correlation measures *shape similarity* (cosine of the angle between the estimated and true peak vectors), not amplitude accuracy. A correlation of 0.793 means the estimated peaks preserve $\sim 79\%$ of the shape structure.
- The remaining $\sim 21\%$ discrepancy comes from (a) amplitude shrinkage due to ℓ_1 bias and (b) baseline leakage distorting the peak shape, especially at the base.
- In practice, $r > 0.95$ is expected for quantitative chromatography. 0.793 would not be acceptable.
- For qualitative analysis (peak detection, retention time identification), 0.793 may be sufficient since the peak *positions* are typically preserved even when amplitudes are distorted.
- The gap from 0.793 to 1.0 is another manifestation of the ℓ_1 leakage problem.

Thesis references: Sec. 5.1 (metric definitions), Table 5.1 (peak correlation).

Q32

The learned parameter analysis (Section 5.2.3) uses a 6-layer model with $N=1024$, but the main results use a 20-layer model with $N=4096$. Are these comparable? Why not analyse the parameters of the actual model you evaluate?

Answer Strategy

Key points:

- They are **not directly comparable**, and this should be acknowledged.
- The 6-layer/ $N=1024$ model was used for the parameter analysis because (a) it is easier to visualise 6 layers and (b) it was an earlier experiment before the full 20-layer model was trained.
- The qualitative pattern (early layers learn coarse separation, later layers learn refinement) is expected to hold across depths, but the specific parameter values would differ.
- Ideally, the parameter trajectories of the 20-layer model should be plotted. This is a gap.
- The analysis still provides insight into the *specialisation phenomenon*, which is the main finding of that section, even if the numerical values are model-specific.

Thesis references: Sec. 5.2.3 (parameter analysis), Sec. 4.2 (model configurations).

7 Baseline Leakage Deep Dive

Q33

You claim leakage is a “fundamental limit of ℓ_1 -based formulations.” But classical BEADS with properly tuned parameters on unnormalised data presumably has lower leakage. Isn’t the leakage just a parameter-tuning problem that you failed to solve?

Answer Strategy

Key points:

- The leakage mechanism is *not* a tuning artefact: it stems from the soft-thresholding operator $\text{soft}(x, \lambda) = \text{sign}(x)(|x| - \lambda)_+$, which *always* shrinks non-zero entries by exactly λ . This bias exists for any $\lambda > 0$.
- Classical BEADS on unnormalised data has the *same* mechanism, but the effect may be less visible because (a) peak amplitudes are 10^3 – 10^6 so the relative shrinkage λ/A is small, and (b) users tune λ to minimise visible artefacts.
- On normalised $[0, 1]$ data, the shrinkage λ/A is relatively larger, making leakage more prominent.
- The deeper point: in *dense-peak regions*, the sparsity assumption fails entirely (peaks overlap, the signal is not sparse), so ℓ_1 is the wrong prior.
- The leakage is thus both a tuning issue (partially) *and* a structural issue (fundamentally, in dense regions).

Thesis references: Sec. 5.5 (leakage analysis), Sec. 3.5 (proximal operator), Eq. 3.11.

Q34

You mention group sparsity as a potential solution to leakage. Can you explain concretely how it would help, and why you did not pursue it?

Answer Strategy

Key points:

- Group sparsity (e.g., group LASSO or $\ell_{2,1}$ penalty) penalises *groups* of adjacent coefficients jointly: $\sum_g \|\mathbf{p}_g\|_2$. Either the entire group is zero or non-zero—no element-wise shrinkage within active groups.
- This directly addresses the leakage mechanism: within an active peak group, amplitudes are *not* shrunk toward zero, preserving peak shape and area.
- Implementation challenge: defining the groups. Peak boundaries are unknown a priori, so the grouping must be adaptive or overlapping (e.g., sliding-window group LASSO).
- The proximal operator for group $\ell_{2,1}$ is a *block* soft-thresholding, which is straightforward to implement in PyTorch.
- Not pursued due to scope and time constraints. This is the highest-priority future work.

Thesis references: Sec. 5.5 (leakage discussion), Sec. 6 (future work).

Q35

Gharbi et al.’s U-HQ method uses a hybrid penalty that avoids ℓ_1 shrinkage bias. You cite it in the background. Why not adopt their approach instead of fighting the ℓ_1 bias with loss terms?

Answer Strategy

Key points:

- U-HQ (Gharbi et al.) uses a non-convex penalty that approximates ℓ_0 : it does not shrink large coefficients, only eliminates small ones. This would indeed avoid leakage.
- The challenge: non-convex penalties lose the convergence guarantees of MM/ISTA. The proximal operator may not be unique, and unrolling a non-convergent algorithm is theoretically questionable.
- Adopting U-HQ would require redesigning the entire unrolling framework: different cost function, different proximal operator, different convergence analysis.
- This is the right direction for future work, but it is a separate thesis.
- The contribution here is establishing *why* the ℓ_1 version fails, which motivates the move to U-HQ-style penalties.

Thesis references: Sec. 2.3 (U-HQ discussion), Sec. 5.5 (leakage analysis), Sec. 6.

Q36

If baseline leakage is truly a fundamental limitation of the ℓ_1 formulation, then why build on BEADS at all? Isn't choosing a flawed base algorithm the core problem?

Answer Strategy

Key points:

- BEADS was chosen because (a) it is the advisor's algorithm, making this a natural extension of the group's research, and (b) it is one of the most widely used baseline estimation methods with a clear optimisation structure suitable for unrolling.
- BEADS is not “flawed”—it is a well-designed convex programme that works well for sparse-peak chromatograms. The limitation arises specifically in the dense-peak regime.
- The thesis demonstrates *exactly where* BEADS breaks down when unrolled. This is a contribution: understanding the limits of an existing method rigorously.
- The negative result is arguably more valuable than a modest positive improvement: it redirects future research toward structural solutions (group sparsity, non-convex penalties).
- “All models are wrong; some are useful.” BEADS is useful; we now know precisely where it stops being useful.

Thesis references: Sec. 1.2 (thesis scope), Sec. 2.3.3 (BEADS), Sec. 5.5 (limitations).

8 Practical / “So What” Questions

Q37

Has LBEADS-NET been validated on even one real chromatographic dataset with known ground truth?

Answer Strategy

Key points:

- **No.** All experiments use synthetic data.
- The fundamental problem: real chromatograms do not have ground-truth decompositions ($\mathbf{b}$, $\mathbf{p}$, ϵ are not separately measurable).
- Semi-synthetic validation is possible: take a known peak standard (e.g., caffeine), subtract a fitted baseline manually, and use the result as “ground truth.” This was not attempted.
- Public chromatographic datasets with ground truth are essentially non-existent. This is a known challenge in the field.
- Real-data validation is the most important next step (Sec. 6).

Thesis references: Sec. 4.1 (synthetic data only), Sec. 6 (future work).

Q38

What is the inference time of LBEADS-NET? If each layer requires a dense $N \times N$ matrix-vector multiply, it might be slower than classical BEADS, which is $O(N)$ per iteration with banded solvers.

Answer Strategy

Key points:

- Inference time is **not reported** in the thesis—this is a gap.
- Each of the $K=20$ layers performs $O(N^2)$ operations due to dense matrix multiplication. For $N=4096$, this is $\sim 20 \times 16\text{M} = 320\text{M}$ flops.
- Classical BEADS with banded solvers is $O(N \cdot b \cdot K)$ where b is bandwidth ($\sim 5\text{--}10$), so $\sim 20 \times 40\text{K} = 800\text{K}$ flops—**400× fewer**.
- On a GPU, the dense matmuls are parallelised, so wall-clock time may be comparable. But on CPU (the typical analytical lab environment), LBEADS-NET is significantly slower.
- With banded implementation, LBEADS-NET would also be $O(N)$ per layer, eliminating the gap.

Thesis references: Sec. 3.4 (implementation), Sec. 6 (future work on banded ops).

Q39

How would a chromatographer actually use LBEADS-NET? They do not have ground-truth baselines for training. Does the model generalise to unseen instrument configurations?

Answer Strategy

Key points:

- Usage scenario: train on synthetic data that covers the expected range of peak shapes, baseline curvatures, and noise levels; deploy on real data.
- This is the standard paradigm for model-based deep learning in domains without ground truth (e.g., MRI reconstruction).
- Generalisation to unseen instruments depends on whether the synthetic data distribution covers the real data distribution. This is not tested.
- Domain adaptation (fine-tuning on a small set of manually corrected real chromatograms) is a natural extension.
- Practically, the user would run LBEADS-NET as a preprocessing step in their chromatography data system (CDS) software, replacing the built-in baseline correction module.

Thesis references: Sec. 4.1 (synthetic data rationale), Sec. 6 (deployment discussion).

Q40

Your hybrid pipeline adds classical BEADS as a post-processing step. If you still need classical BEADS at the end, what did unrolling actually buy you?

Answer Strategy

Key points:

- The hybrid pipeline uses LBEADS-NET to provide an *initial decomposition* that is closer to the true solution than the default initialisation ($\mathbf{b}_0 = 0$). Classical BEADS then refines from this warm start.
- The benefit: BEADS converges in fewer iterations from a good initialisation, and the learned parameters provide signal-adaptive regularisation weights that BEADS alone cannot.
- Concede: if classical BEADS with proper tuning achieves similar results, the unrolling step adds complexity without clear benefit.
- The honest framing: the hybrid pipeline is a *fallback strategy* when LBEADS-NET alone is insufficient, not the primary contribution.
- The primary contribution is the understanding of leakage, not the pipeline itself.

Thesis references: Sec. 3.9 (hybrid pipeline), Sec. 5.2 (hybrid results).

9 Writing & Completeness

Q41

Your abstract is Lorem Ipsum. Your conclusions chapter is Lorem Ipsum. Your ablation tables have all dashes. Several sections are marked as placeholders. Is this thesis actually complete?

Answer Strategy

Key points:

- **Do not get defensive.** Acknowledge that these sections are incomplete and will be finalised before the official submission deadline.
- Emphasise that the *technical content* (Chapters 2–5) is complete and the defence focuses on the technical contribution.
- Provide a timeline: abstract and conclusions will be written after the defence, incorporating committee feedback.
- For the ablation tables: state which experiments are planned and the expected timeline.
- This is a common situation for defences that occur before the final manuscript submission.

Thesis references: Abstract, Chapter 6 (conclusions), Tables 5.3–5.4.

Q42

Several cross-references in your document show “Chapter ??” (pages 43, 56, 68). Have you compiled and checked the final PDF?

Answer Strategy

Key points:

- Acknowledge the broken references. These are caused by missing `\label{}` tags or mismatched `\ref{}` keys.
- State that a full compilation pass with `latexmk` (multiple runs to resolve references) will be done before submission.
- This is a mechanical fix, not a content issue.
- Have a plan: `grep` for “??” in the compiled PDF, trace each to the source `\ref{}` command, and fix.

Thesis references: Throughout document (cross-references).

Q43

You say “twelve-term composite loss” in Section 3.7 and Table 3.3, but your Introduction says “11-term.” Which is correct?

Answer Strategy

Key points:

- The correct count is **twelve**. Section 3.7 and Table 3.3 enumerate all twelve terms explicitly.
- The Introduction’s “11-term” is a typo from an earlier version of the loss function before the twelfth term (likely the envelope constraint) was added.
- This will be corrected in the Introduction before submission.
- Having the specific section and table to point to (Sec. 3.7, Table 3.3) provides a definitive answer.

Thesis references: Sec. 1.2 (Introduction, “11-term”), Sec. 3.7 (“twelve-term”), Table 3.3.

10 Future Work & Big Picture

Q44

If you had 6 more months, what would you do first and why?

Answer Strategy

Key points (prioritised):

1. **Group sparsity penalty** (2 months): replace ℓ_1 with $\ell_{2,1}$ or overlapping group LASSO. This directly addresses the leakage ceiling and is the highest-impact change.
2. **Banded matrix implementation** (1 month): exploit the sparsity structure of $\mathbf{H}$ and $\mathbf{D}$ to reduce memory from $O(N^2)$ to $O(N)$ and enable $N=65536+$ signals.
3. **Real data validation** (1 month): obtain chromatographic data from a collaborating lab, create semi-synthetic ground truth, and evaluate.
4. **arPLS/airPLS baseline comparison** (2 weeks): implement and tune arPLS on the existing synthetic dataset.
5. **Learnable f_c** (2 weeks): parameterise the highpass filter as a learnable FIR filter initialised from the Butterworth design.
6. **Complete ablations** (2 weeks): fill Tables 5.3 and 5.4.

Thesis references: Sec. 6 (future work), Sec. 5.5 (leakage analysis).

Q45

Could the LBEADS-NET approach generalise to other signal decomposition problems—Raman spectroscopy, NMR, mass spectrometry?

Answer Strategy

Key points:

- **Yes, in principle.** Any problem where the signal decomposes as $\mathbf{y} = \mathbf{b} + \mathbf{p} + \boldsymbol{\epsilon}$ with a smooth baseline and sparse peaks is a candidate.
- **Raman spectroscopy:** fluorescence background is smooth and broadband, peaks are narrow Lorentzian—very similar structure. BEADS-like methods are already used (Zhang et al., 2010).
- **NMR:** baseline distortion from phase errors and receiver artefacts. The peak model is Lorentzian with known analytic form.
- **Mass spectrometry:** chemical noise creates a baseline, but the signal is often discrete (centroid data), which changes the problem structure.
- The unrolling framework is modular: replace the peak prior (Lorentzian instead of bi-Gaussian) and the filter design (adjust f_c for the spectral content), and retrain.
- The ℓ_1 leakage limitation would apply equally in all these domains—the proposed group sparsity fix would also be needed.

Thesis references: Sec. 2.1 (signal model), Sec. 6 (future work).

Q46

What would it take to make f_c learnable? Sketch the approach at the whiteboard.

Answer Strategy

Key points (sketch for the whiteboard):

1. **Current:** f_c enters through `scipy.signal.butter(d, f_c)` $\rightarrow$ IIR coefficients $\rightarrow$ dense matrix $\mathbf{H}$. This pipeline is not differentiable.
2. **Approach 1—Differentiable IIR:** Place Butterworth poles at $p_k = e^{j(\pi/2 \pm \Delta\theta_k)}$ where $\Delta\theta_k = g(f_c)$. Construct the transfer function $H(z) = \prod_k (1 - p_k z^{-1})$ in PyTorch. Apply via `torch.linalg.solve_triangular` (causal IIR). Gradients flow through $f_c \rightarrow p_k \rightarrow H(z) \rightarrow \mathbf{H}\mathbf{y}$.
3. **Approach 2—Learnable FIR:** Design an FIR filter of order M initialised from the Butterworth impulse response truncated at M taps. Make the tap weights $h_0, \dots, h_M$ learnable. Apply via `torch.nn.functional.conv1d`. This is simpler but loses the sharp IIR rolloff.
4. **Approach 3—Frequency-domain:** Multiply $\mathbf{Y}(f)$ by a learnable frequency response $|\hat{H}(f)|$ parameterised by f_c and filter order. Invert via IFFT. Fully differentiable but introduces circular convolution artefacts.
5. **Recommended:** Approach 2 (FIR) for simplicity and robustness, with $M \sim 64$ taps.

Thesis references: Sec. 3.4 (current filter construction), Sec. 6 (future work).

11 Killer Questions — The Ones That Hurt

Warning

These are the questions designed to make you uncomfortable. They target circular reasoning, contribution validity, mathematical rigour, and the “so what” factor. If you can handle these, you can handle anything.

Q47

Strip away the loss engineering that your own results show does not work. What remains? You unrolled BEADS into ISTA layers with scalar parameters—that is just LISTA applied to a different cost function. Where, exactly, is the novelty?

Answer Strategy

Key points:

- Acknowledge the similarity to LISTA, but clarify the differences: (a) LISTA learns full matrices $\mathbf{W}_e, \mathbf{S}$; LBEADS-NET fixes the operators and learns only scalars. (b) LISTA targets sparse coding; LBEADS-NET targets a *three-component* signal decomposition with frequency-domain separation (baseline + peaks + noise).
- The novelty is application-specific: *no prior work has unrolled the BEADS algorithm*. The ISTA reformulation of the MM update (replacing CG inner solve with proximal gradient) is a design contribution.
- The identification and systematic characterisation of baseline leakage as a fundamental ℓ_1 limitation in this architecture is novel.
- The 12-term anti-leakage loss design (even though it fails to break the BLI floor) and the curriculum strategy are original engineering contributions.
- Concede: the theoretical novelty is modest. This is an *applied* thesis.

Thesis references: Sec. 2.3.2 (LISTA), Sec. 3.2–3.3 (ISTA reformulation), Sec. 1.2 (contributions).

Q48

You train on synthetic data and test on synthetic data generated by the same code with the same distributions. This is a closed loop. You have proven nothing about real chromatography. How do you respond?

Answer Strategy

Key points:

- This is correct, and it is a fundamental limitation.
- The synthetic data was calibrated against 8 real chromatograms (Sec. 4.1)—peak heights 10–2600+, widths FWHM 3–370+, 29–58 peaks per signal—but the test set is still synthetic.
- The thesis contribution is *algorithmic*: demonstrating unrolling and characterising leakage. The synthetic evaluation framework controls for confounding variables that real data would introduce.
- All supervised deep learning for baseline correction (Kensert et al., Chen et al., Han et al.) uses synthetic training data—this is standard practice because ground truth does not exist for real chromatograms.
- The honest response: real-data validation is the single most important missing piece and the top priority for future work.

Thesis references: Sec. 4.1 (data generation), Sec. 5.6 (real chromatogram discussion).

Q49

Your standard deviations are enormous. Peak MSE is $9.30 \pm 4.7 \times 10^{-3}$. That means some signals have MSE of 4.6×10^{-3} and others have 14.0×10^{-3} —a $3\times$ range. Is your model actually learning anything consistent, or is it just averaging over easy and hard cases?

Answer Strategy

Key points:

- The high variance is expected: the test set deliberately includes signals with very different peak densities (20–55 peaks), baseline shapes (spline, exponential, mixed), and noise levels ($\sigma_w \in [2, 6]$).
- A signal with 20 well-separated peaks and low noise is genuinely *easier* than one with 55 overlapping peaks and high noise. The variance reflects problem difficulty, not model inconsistency.
- To verify: stratify results by peak density (sparse vs dense) and report separately. This was not done but should be.
- The *relative* improvement over classical BEADS is consistent across signals (the std of the improvement would be more informative than the std of the raw metric).
- Concede: reporting only mean $\pm$ std is insufficient. Median, IQR, and per-signal scatter plots would be more informative.

Thesis references: Table 5.1 (standard deviations), Sec. 4.1 (data diversity).

Q50

*You say LBEADS-NET has “only 100 learnable parameters.” But the dense matrices **A**, **B**, and **L** are each $N \times N = 4096^2 \approx 16.7$ million entries. These are frozen, not absent. The model capacity is dominated by these fixed operators. Isn’t “100 parameters” misleading?*

Answer Strategy

Key points:

- This is a valid nuance. The *learnable* parameter count is 100 scalars. The *total model capacity* includes the fixed filter matrices, which encode substantial prior knowledge.
- However, the fixed matrices are determined entirely by two hyperparameters (f_c and d) and the signal length N . They are not “learned from data”—they are analytically derived from the Butterworth filter design.
- The analogy: a CNN with a fixed Gabor filter bank and 100 learnable classification weights also has “100 learnable parameters” even though the Gabor filters encode structure.
- The “100 parameters” claim is about *what backpropagation can adjust*, not about the total information content of the model. This distinction should be made clearer in the thesis.
- The correct framing: “100 learnable parameters, with fixed structural operators derived from the BEADS formulation.”

Thesis references: Sec. 3.4 (architecture), Table 3.1 (parameters per layer).

Q51

Why not just use Bayesian optimisation to tune the 6 classical BEADS parameters on a validation set? That would be simpler than unrolling, fully interpretable, and might work just as well.

Answer Strategy

Key points:

- Bayesian optimisation (BO) over $\{f_c, d, r, \lambda_0, \lambda_1, \lambda_2\}$ on a validation set is a perfectly reasonable baseline. It was not compared against, and it should have been.
- Key difference: BO finds a *single global* parameter set optimised for the validation distribution. LBEADS-NET learns *per-layer* parameters that specialise across depth (Sec. 5.5)—this is fundamentally richer.
- BO-tuned BEADS would still suffer from the same ℓ_1 leakage in dense-peak regions (the leakage is structural, not a tuning issue). But BO would likely achieve better results than the “unfair” default-parameter comparison.
- Concede: BO-tuned BEADS is the missing comparison that would isolate the benefit of *unrolling* from the benefit of *better parameters*.
- This is arguably the most important missing experiment.

Thesis references: Sec. 4.5 (comparison methods), Sec. 5.1 (results).

Q52

Your three-stage curriculum training is just a manually designed training schedule. You replaced BEADS’s 6 hand-tuned parameters with 12 hand-tuned loss weights plus a hand-designed 3-stage curriculum with hand-picked epoch budgets. How is this reducing human design effort?

Answer Strategy

Key points:

- This is a fair and damaging critique. Acknowledge it.
- The key distinction: the manual design effort is a *one-time cost* during model development. Once trained, the model requires *zero* user input at inference time. Classical BEADS requires per-signal tuning at every use.
- The curriculum and loss weights are meta-level design decisions analogous to choosing a network architecture in deep learning—they are set by the model developer, not the end user.
- Concede: the total design effort for LBEADS-NET (12 loss weights, 3 curriculum stages, epoch budgets, etc.) is *greater* than tuning BEADS for a single signal. The payoff is generalisation across signals.
- Automated hyperparameter optimisation (e.g., learning the loss weights via meta-learning) would strengthen this claim and is a natural extension.

Thesis references: Sec. 3.6 (curriculum), Table 3.2 (stage configuration), Sec. 3.7 (loss weights).

Q53

Gharbi et al. already showed in 2024 that ℓ_1 -based unrolled networks exhibit shrinkage bias in chromatographic signal recovery. You cite their work. Your “fundamental insight” about baseline leakage is a confirmation of their finding, not a new discovery. What do you actually add beyond their result?

Answer Strategy

Key points:

- Gharbi et al. (2024) observed that ℓ_1 -based unrolled methods (U-PD, U-ISTA) produce biased peak intensities compared to their U-HQ hybrid penalty. This is indeed the same underlying phenomenon.
- What LBEADS-NET adds: (a) **Quantification**: the BLI and PBER metrics provide the first *numerical characterisation* of the leakage magnitude ($\text{BLI} \approx 0.59$). (b) **Systematic mitigation attempt**: 12 anti-leakage loss terms, progressive experiments, and the proof that loss engineering cannot break the floor. (c) **Different algorithm**: BEADS (MM-based) vs Gharbi’s PD/ISTA/HQ formulations. (d) **Curriculum training**: novel for this domain.
- Concede: the conceptual finding is not new. The contribution is the *depth of analysis* and the specific characterisation for the BEADS architecture.
- Frame as: “We independently confirmed and quantified a phenomenon that Gharbi et al. identified qualitatively.”

Thesis references: Sec. 2.3.5 (Gharbi et al.), Sec. 5.1.2 (BLI analysis), Sec. 1.3 (contributions).

Q54

At the end of the day: your best model has 50% area error, 0.79 peak correlation, and 0.59 BLI. A practising chromatographer would never deploy this. Your loss engineering does not fix leakage. Your ablation tables are empty. What exactly is the contribution to knowledge?

Answer Strategy

Key points (this is THE critical question—have this answer cold):

- **Contribution 1**: First unrolling of the BEADS algorithm. Demonstrated that BEADS can be reformulated as ISTA layers, trained end-to-end, and applied without per-signal parameter tuning.
- **Contribution 2**: Identification and quantification of the ℓ_1 leakage floor ($\text{BLI} \approx 0.59$) as a *fundamental architectural limit* that loss engineering cannot overcome. This is a negative result with clear implications for the field.
- **Contribution 3**: The failure mode catalogue—CG gradient vanishing, non-learnable f_c , train/inference mismatch, softplus artefacts—provides practical guidance for anyone attempting algorithm unrolling.
- **Contribution 4**: Emergent parameter specialisation across layers, demonstrating that unrolled networks learn non-uniform iteration schedules that classical algorithms cannot express.
- **The framing**: this is a *methodological exploration* and *negative result study*, not a deployment-ready tool. The value is in *what we learned* about unrolling sparsity-based algorithms.

Thesis references: Sec. 1.3 (contributions), Sec. 5.7 (summary of findings).

Q55

Your advisor invented BEADS. How do we know this thesis is not just an exercise in making the advisor’s algorithm look relevant? Where is the objectivity?

Answer Strategy

Key points:

- The thesis is arguably *harder* on BEADS than an independent researcher would be: it identifies three failure modes of BEADS (sparsity violation, filter bandwidth mismatch, spatially uniform regularisation), demonstrates that the ℓ_1 prior is fundamentally limited in dense-peak regimes, and shows that unrolling inherits these limitations.
- A thesis that just praised BEADS would be weaker. The negative findings (leakage floor, over-regularisation, CG gradient collapse) are honest scientific reporting.
- BEADS was chosen because it has the cleanest structure for unrolling (explicit MM iterations, scalar parameters, joint peak+baseline estimation). This is a technical motivation, not a promotional one.
- The thesis *motivates moving beyond BEADS* (group sparsity, U-HQ penalties) as the primary future direction.

Thesis references: Sec. 2.2.1.5 (BEADS failure modes), Sec. 5.1.2 (leakage analysis), Sec. 5.7 (honest summary).

Q56

Can you derive the soft-thresholding operator on the board right now? Show me that it is the proximal operator of the ℓ_1 norm, and explain why it causes shrinkage bias.

Answer Strategy

Whiteboard derivation (MUST be prepared):

1. The proximal operator: $\text{prox}_{\lambda\|\cdot\|_1}(\mathbf{v}) = \arg \min_{\mathbf{x}} \frac{1}{2}\|\mathbf{x} - \mathbf{v}\|^2 + \lambda\|\mathbf{x}\|_1$.
2. The problem separates element-wise: $\min_{x_i} \frac{1}{2}(x_i - v_i)^2 + \lambda|x_i|$.
3. Take subgradient, set to zero: $(x_i - v_i) + \lambda \cdot \text{sign}(x_i) = 0$.
4. Three cases: $v_i > \lambda \Rightarrow x_i = v_i - \lambda$; $v_i < -\lambda \Rightarrow x_i = v_i + \lambda$; $|v_i| \leq \lambda \Rightarrow x_i = 0$.
5. This is $\mathcal{S}_\lambda(v_i) = \text{sign}(v_i) \max(|v_i| - \lambda, 0)$.
6. **The bias:** for any non-zero true signal $x^* > 0$, $\mathcal{S}_\lambda(x^*) = x^* - \lambda < x^*$. The estimate is *always* shrunk by exactly λ . In dense-peak regions where many entries are non-zero, this shrinkage accumulates.

Thesis references: Eq. 3.14 (asymmetric soft-thresholding), Sec. 3.2.4 (proximal step).

Q57

Your masked baseline loss (Term 5) supervises the baseline only where peaks are absent. But at inference time, you do not know where peaks are. The model was trained with ground-truth peak masks that it will never have in deployment. Isn't the model learning to rely on information it cannot access?

Answer Strategy

Key points:

- The masked loss provides a *training signal*, not an inference-time input. The network does not receive the mask at test time—it receives only $\mathbf{y}$.
- The training objective teaches the network to produce baselines that are accurate in non-peak regions. At inference, the network generalises this learned behaviour to unseen signals.
- This is standard supervised learning: the ground truth is used to compute the loss during training but is not available at test time. All supervised losses “rely on information the model cannot access at inference.”
- The concern would be valid if the mask were an *input* to the network. It is not—it only shapes the gradient.
- Potential issue: if the peak mask definition ($\gamma = 0.02$ threshold) is very different between training and real data, the learned baseline behaviour may not transfer.

Thesis references: Sec. 3.7.3 (masked baseline loss), Eqs. 3.27–3.29.

Q58

You claim “interpretability” as a key advantage. But can you actually diagnose why LBEADS-NET fails on a specific signal by inspecting the learned parameters? Show me how.

Answer Strategy**Key points:**

- Partial yes: the per-layer parameters have physical meaning. If $\lambda_0^{(k)}$ is very large in early layers, one can diagnose “early layers are over-thresholding small peaks.” If $r^{(k)}$ is low, “insufficient asymmetric penalisation of negative artefacts.”
- More concretely: one can run the signal through each layer sequentially, inspecting the intermediate $\mathbf{x}^{(k)}$ and $\mathbf{f}^{(k)}$ at each stage. This “layer-by-layer decomposition” is impossible with a black-box U-Net.
- However, with $K=20$ layers and 5 parameters each, the interaction effects are complex. “Interpretable” does not mean “simple to diagnose.”
- The emergent specialisation (Sec. 5.5) demonstrates that the parameters *are* interpretable post-training. But using them for per-signal failure diagnosis is an unsolved practical problem.
- Honest answer: “more interpretable than a black box” is correct, but “fully diagnosable” is an overstatement.

Thesis references: Sec. 5.5 (parameter analysis), Sec. 3.4.3 (layer chaining).

Q59

You exploit dense $O(N^2)$ matrices when the original BEADS uses $O(N)$ banded solvers. An undergraduate could implement banded matrix-vector products in PyTorch. Why didn’t you?

Answer Strategy**Key points:**

- Concede: banded matrix operations are straightforward in theory. The challenge is integrating them cleanly with PyTorch’s autograd for backpropagation.
- The highpass filter involves $\mathbf{H} = \mathbf{B}\mathbf{A}^{-1}$, where $\mathbf{A}$ is banded. The forward pass requires solving $\mathbf{A}\mathbf{z} = \mathbf{v}$ (banded system solve). The *backward* pass requires differentiating through this solve, which PyTorch does not natively support for banded systems.
- Options: (a) use `torch.linalg.solve_triangular` with LU-factored $\mathbf{A}$, (b) write a custom autograd function for banded solves, (c) use implicit differentiation.
- All of these require more engineering effort than converting to dense, which was the pragmatic choice given the thesis timeline.
- This is the single most important engineering improvement for practical deployment.

Thesis references: Sec. 3.2.2 (filter construction), Sec. 3.4 (dense implementation).

Q60

ADMM-Net is the standard framework for unrolling constrained optimisation problems. Why did you not use ADMM? The baseline non-negativity constraint and the frequency separation are natural ADMM constraints.

Answer Strategy

Key points:

- ADMM decomposes the problem into sub-problems that handle different constraints/penalties separately, linked by dual variables (Lagrange multipliers). This would naturally separate peak sparsity, baseline smoothness, and frequency separation into distinct ADMM updates.
- BEADS is formulated as an unconstrained MM problem, not as a constrained optimisation. Converting to ADMM would require reformulating the cost function with explicit constraints, which changes the algorithm being unrolled.
- The thesis's goal is to unroll *BEADS specifically*, preserving its structure. ADMM-based unrolling would be a different method.
- ADMM introduces additional hyperparameters (penalty parameter ρ , dual variable update rate) that would also need to be learned per layer.
- Concede: ADMM unrolling is a promising alternative approach that should be explored.

Thesis references: Sec. 2.3.4 (ADMM-Net survey), Sec. 3.2 (ISTA choice rationale).

Q61

Your model is trained at $N=4096$ and tested at $N=4096$. Real chromatograms can have $N=10^5$ to 10^6 points. What happens when you apply this model to a different signal length? Does it even work?

Answer Strategy

Key points:

- **It does not work directly.** The dense matrices **A**, **B**, **L** are precomputed at construction time for a fixed N . A different signal length requires reconstructing these matrices (which changes their spectral properties) and re-registering the buffers.
- The 100 learned scalar parameters are *independent of N* —they can be transferred to any signal length. Only the filter matrices need to be recomputed.
- This is a strength of the architecture: the learned parameters are signal-length-agnostic. The filter construction adapts automatically via `BAfilt(d, fc, N_new)`.
- However, **this was not tested**. The normalised cutoff $f_c=0.006$ has a different physical meaning at different N (the period $1/f_c \approx 167$ samples is fixed, so longer signals have proportionally more baseline variation relative to the filter's passband).
- With banded implementation, arbitrary N would be trivial. With dense matrices, $N=10^5$ requires ~ 80 GB per matrix—infeasible.

Thesis references: Sec. 3.4 (architecture), Sec. 3.2.2 (filter parameterisation).

Q62

You initialise all layers with identical parameters and let training differentiate them. But you only train for 30 epochs (300 gradient steps). With 100 parameters to differentiate in 300 steps, haven't the parameters barely moved from their initialisation? How much specialisation can actually emerge?

Answer Strategy

Key points:

- The 30-epoch / 300-step budget is indeed very tight for 100 parameters. This is why the more complex loss configurations (Exp. 1.5) fail—they need more steps.
- However, the progressive experiments show meaningful learning: Peak MSE drops from 21.27 (classical) to 9.30 (Exp. 1.3) in 300 steps. The parameters have clearly moved.
- The parameter analysis in Sec. 5.5 uses a different model (6 layers, 100 epochs = 1000 steps) where specialisation is more pronounced.
- Concede: the 30-epoch budget was a practical constraint (training time on CPU), not an optimised choice. Longer training would likely improve all configurations, especially the complex ones.
- This is precisely why the curriculum training (Sec. 3.6) exists: it provides a *structured path* through parameter space that achieves more in fewer steps than random initialisation.

Thesis references: Table 4.2 (training config), Sec. 5.5 (parameter analysis), Sec. 3.6 (curriculum).

Q63

Write the BEADS cost function on the board. Now take the gradient with respect to $\mathbf{x}$. Now show me how one MM iteration relates to one ISTA step. Where does the approximation enter?

Answer Strategy

Whiteboard derivation (MUST be prepared):

1. BEADS cost: $J(\mathbf{x}) = \frac{1}{2}\|H(\mathbf{y} - \mathbf{x})\|^2 + \lambda_0 \sum_n \theta(x_n) + \lambda_1 \sum_n \phi(\Delta x_n) + \lambda_2 \sum_n \phi(\Delta^2 x_n)$.
2. Split into smooth + non-smooth: $J = \underbrace{J_{\text{smooth}}}_{\text{data fidelity + smoothness}} + \underbrace{J_{\text{non-smooth}}}_{\lambda_0 \sum \theta(x_n)}$.
3. $\nabla J_{\text{smooth}} = -H^T H(\mathbf{y} - \mathbf{x}) + \lambda_1 D_1^T \psi(D_1 \mathbf{x}) + \lambda_2 D_2^T \psi(D_2 \mathbf{x})$ where $\psi(z) = z/(|z| + \epsilon)$.
4. **MM step:** majorise the non-quadratic terms, solve the resulting banded linear system exactly. This gives *exact* majoriser minimisation.
5. **ISTA step:** take one gradient step on J_{smooth} : $\mathbf{x}_{\text{update}} = \mathbf{x}^{k-1} + \eta \nabla J_{\text{smooth}}$, then apply proximal operator for θ : $\mathbf{x}^k = \mathcal{S}_{\text{asym}}(\mathbf{x}_{\text{update}}; \lambda_0 \eta, r)$.
6. **The approximation:** ISTA replaces exact MM minimisation with a single gradient step + proximal. It is less accurate per iteration but *much simpler* to differentiate.

Thesis references: Eq. 2.1 (BEADS cost), Algorithm 1 (MM), Algorithm 2 (ISTA layer), Sec. 3.3.

Q64

Your v5 model uses clamped exponential parameterisation, but Section 3.2.1 says “No clamping is applied—the exponential function alone guarantees positivity.” Which is it? Does the 6-layer parameter analysis (Table 5.5) show clamped parameters (marked with *)? These statements contradict each other.

Answer Strategy

Key points:

- This is a genuine inconsistency in the thesis.
- Section 3.2.1 describes the v5 architecture (the final model) where parameters use unclamped exponential parameterisation: $\lambda = \exp(\theta)$, no bounds.
- Table 5.5 shows parameters from a *different model* (6-layer, $N=1024$, curriculum-trained) that uses clamped parameterisation (parameters marked with * hit clamp boundaries).
- The two models have different parameterisation schemes. This is not made clear in the thesis and creates confusion.
- Fix: add a note in Sec. 5.5 that the 6-layer model uses a different (earlier) parameterisation with clamps, and clarify that the main 20-layer model uses unclamped exponential.

Thesis references: Sec. 3.2.1 (unclamped), Table 5.5 (clamped * markers).

Q65

You claim the per-layer independence enables “emergent parameter specialisation.” But with 100 parameters trained on 160 signals for 30 epochs, how do you distinguish “specialisation” from “overfitting to the training set”? What would the parameters look like on a different random seed?

Answer Strategy

Key points:

- Valid concern. The monotonic increase in λ_0 across layers could be a genuine pattern or a training artefact.
- To distinguish: train multiple models with different random seeds and check whether the same qualitative pattern (increasing sparsity, non-monotonic asymmetry, stable step size) appears. This was not done.
- Evidence against overfitting: the test-set metrics improve consistently across the progressive experiments, suggesting the learned parameters generalise. If the model were overfitting, we would expect test performance to diverge from training performance.
- The specialisation pattern is also *physically sensible*: it matches the behaviour of iterative thresholding algorithms (coarse-to-fine refinement). Overfitting would produce arbitrary or noisy parameter values.
- Concede: multi-seed experiments are needed to confirm the robustness of the specialisation pattern.

Thesis references: Sec. 5.5 (parameter analysis), Table 5.5 (learned values).

Q66

You have 100 learnable scalars per layer— $\lambda_0, \lambda_1, \lambda_2, r, \eta$ —but you leave the single most important parameter fixed: the cutoff frequency f_c . f_c literally defines the boundary between “peak” and “baseline.” You admit it contributes to leakage through filter bandwidth mismatch. You even list three concrete approaches to make it learnable in Section 3.5. So why didn’t you just... do it? You had the entire thesis to implement a differentiable IIR filter or a learnable FIR. What stopped you?

Answer Strategy

Key points (this one stings—be honest and precise):

- **The technical barrier is real but not insurmountable.** The filter matrices A and B are constructed by `scipy.signal` routines that are opaque to PyTorch’s autograd. The chain $f_c \rightarrow \mathbf{a}, \mathbf{b} \rightarrow A, B \rightarrow L = I - BA^{-1} \rightarrow \hat{\mathbf{f}}$ passes through non-differentiable SciPy operations, so $\partial \mathcal{L} / \partial f_c = 0$ identically. This is not a gradient-vanishing problem—there is *no gradient at all*.
- **Why not re-implement in PyTorch?** The Butterworth pole placement *is* closed-form: $t = ((1 - \cos \omega_c) / (1 + \cos \omega_c))^d$, and the coefficient vectors $\mathbf{a}, \mathbf{b}$ are polynomial functions of t . In principle, all of this can be written in pure PyTorch tensor ops, making f_c differentiable. **The reason it was not done:** the current architecture stores A, B , and L as *dense* $N \times N$ matrices. Making f_c learnable means these matrices must be *recomputed at every forward pass* (since f_c changes with each gradient step). Rebuilding three 4096×4096 dense matrices every forward pass—and backpropagating through N^2 entries—is computationally prohibitive. The engineering prerequisite is banded matrix support, which was itself not implemented.
- **The dependency chain:** learnable f_c requires differentiable filter construction, which requires either banded matrix ops in PyTorch (not implemented) or accepting $O(N^2)$ recomputation per forward pass (infeasible at $N=4096$). Solving either prerequisite was a thesis-scale effort on its own.
- **The FIR alternative:** replacing the IIR Butterworth with a learnable FIR filter of fixed order M (initialized from the Butterworth impulse response) would bypass the dense matrix issue entirely—`torch.nn.functional.conv1d` handles everything. This is approximately 50 lines of code and should have been tried. Concede this directly.
- **The honest answer:** the dense matrix decision early in development created a technical debt that blocked learnable f_c . The FIR workaround was identified late and deprioritized relative to the loss engineering experiments. In hindsight, learnable f_c via FIR would have been higher-impact than the 12-term loss function.
- **Impact on leakage:** f_c contributes to leakage through Failure Mode 2 (filter bandwidth mismatch, Sec. 2.2.1.5). Broad peaks with spectral content below f_c have their energy captured by L and attributed to the baseline. A learnable f_c —or better, a *per-layer* f_c^k —could adapt the frequency boundary to the signal, potentially reducing this component of leakage. However, it would not address Failure Mode 1 (sparsity violation), which is the dominant cause. So learnable f_c is necessary but not sufficient.

Thesis references: Sec. 3.5 (non-learnable f_c and autograd boundary), Sec. 2.2.1.5 (Failure Mode 2: filter bandwidth mismatch), Sec. 3.2.2 (filter construction), Sec. 6 (future work).

12 Known Inconsistencies & Issues to Fix Before Defense

CRITICAL: Fix Before Defense

The following issues have been identified in the current thesis draft. Each should be resolved before the defense to avoid uncomfortable questions.

#	Issue	Location	Fix
1	Abstract is Lorem Ipsum placeholder text	Abstract page	Write the real abstract
2	Conclusions chapter is Lorem Ipsum placeholder	Chapter 6	Write real conclusions
3	Dedication has “YOUR DEDICATION (1-2 LINES)” placeholder	Dedication page	Write dedication or remove
4	Broken cross-references showing “Chapter ??”	pp. 43, 56, 68	Fix \label/\ref pairs
5	University ID and NetID are placeholders (“N#”, “NetID”)	Title/signature pages	Insert real ID numbers
6	Ablation tables 5.3 and 5.4 have all dashes	Tables 5.3–5.4	Run experiments or remove
7	Sections 5.3 (Qualitative Examples) and 5.4 (Training Dynamics) are placeholder	Sec. 5.3–5.4	Write or mark as future work
8	Advisor title inconsistency: “Assistant Prof.” vs “Professor”	Abstract vs. approval page	Verify correct title, unify
9	Loss term count: “11-term” in Intro vs “twelve” in Sec. 3.7	Sec. 1.2 vs Sec. 3.7	Correct Intro to “twelve-term”
10	Peak model mismatch: EMG in Sec. 2.1.4 vs bi-Gaussian in Sec. 4.1.1	Sec. 2.1.4 vs 4.1.1	Add justification in Sec. 4.1.1

Table 1: Known issues in the current thesis draft, ordered by severity.

Priority Actions

- P0** Items 1, 2, 5: These will be immediately noticed by any committee member opening the document.
- P1** Items 3, 4, 8, 9: Visible quality issues that suggest the document was not carefully reviewed.
- P2** Items 6, 7, 10: Content gaps that may or may not be raised depending on committee member interests.

General Defense Advice

Guiding Principles for the Defense

1. **Own the negative results.** The BLI plateau at 0.59 and the over-regularisation finding (Exp. 1.5) are *contributions*, not failures. Present them as deliberate investigations. “We tried X, and it did not work because of Y” is a perfectly valid scientific finding.
2. **Know what you do not know.** When a question targets a gap (e.g., “why no arPLS comparison?”), say: “That is a limitation of this work. Here is what I would do if I had more time.” Do not bluff.
3. **Redirect to the architecture.** Many attack questions (area error, BLI, peak correlation) have the same root cause: ℓ_1 shrinkage bias. Learn the shrinkage bias argument cold and redirect to it.
4. **Prepare the whiteboard.** Be ready to derive: (a) the BEADS cost function (Eq. 2.24), (b) the ISTA proximal step (Eq. 3.9), (c) the soft-thresholding operator and its bias, (d) why untied parameters break MM convergence.
5. **Credit the advisor.** Professor Selesnick co-authored BEADS. When discussing the algorithm, frame the work as a natural evolution of the group’s research programme.
6. **Time management.** Aim for 25–30 minutes of presentation, leaving 30–45 minutes for questions. For each question, answer in 60–90 seconds. If the committee wants more detail, they will ask.
7. **Do not read from this document during the defense.** Internalise the key points. The goal is fluency, not recitation.

*Prepared April 2026.
Good luck, Saad.*

Glossary of Key Terms

Terms Defined in the Context of This Thesis

This glossary defines terms as they are used specifically within the LBEADS-NET thesis. Definitions are scoped to the chromatographic baseline-correction context and may differ from their broader usage in other fields.

ℓ_1 Leakage Limit

The empirically observed floor on the Baseline Leakage Index ($\text{BLI} \approx 0.59$) that persists across *all* LBEADS-NET loss configurations (Experiments 1.1–1.5). The limit arises because the ℓ_1 -based soft-thresholding operator at the core of every ISTA layer shrinks *every* non-zero signal value by exactly λ (the effective threshold). In dense-peak regions where the sparsity assumption is violated, this shrinkage accumulates: the total missing peak energy is proportional to $\lambda \times (\text{number of non-zero samples})$, and this energy is absorbed into the baseline estimate via the low-pass filter. Because the shrinkage is a structural property of the proximal operator—not a tunable weight—no amount of loss engineering (anti-leakage penalties, asymmetric baseline loss, envelope constraints, etc.) can push the BLI below this floor. The limit generalises to *any* unrolled network that inherits ℓ_1 -based sparsity priors, as independently confirmed by Gharbi et al. (2024) in their work on unrolled sparse-recovery architectures. *Thesis refs: Sec. 5.1.2, Table 5.2, Sec. 1.3 (Contribution 3)*

Loss Weight Reference (α values)

Complete α Reference

The total loss is $\mathcal{L}_{\text{total}} = \sum_{i=1}^{12} \alpha_i \mathcal{L}_i$. Each α_i is a *fixed scalar weight set once before training*—it is **not** learned by backpropagation. The α values define the training objective; the 100 per-layer parameters $(\lambda_0^k, \lambda_1^k, \lambda_2^k, r^k, \eta^k)$ are what the optimizer actually adjusts. Different experiments use different α configurations (see Table 4.3).

#	α name	Range used	Stage	What it controls & why this value
1	$\alpha_{\text{mse}} = 1.0$	1.0 (fixed)	A	Reconstruction fidelity. Weight of the Huber loss between $\hat{\mathbf{x}}$ and $\mathbf{x}$. Always 1.0 because it is the anchor—all other weights are defined <i>relative</i> to reconstruction. Active in all three curriculum stages.
2	α_{ℓ_1}	0.005–0.01	B	ℓ_1 sparsity on peaks. Encourages most peak samples to be zero. Kept very low (≤ 0.01) because aggressive ℓ_1 <i>causes</i> baseline leakage by over-shrinking dense peaks. At 0.01 it suppresses false peaks in flat regions without excessive shrinkage. Exp. 1.4 (v5) uses 0.01; Exp. 1.5 (v7) reduces to 0.005 to accommodate stronger anti-leakage terms.
3	α_{tv}	0.005–0.01	B	Total variation on peaks. Promotes piecewise-constant peak shapes with sharp edges. Same low weight as ℓ_1 for the same reason: TV also causes shrinkage in dense regions. Together with ℓ_1 , these two terms are both the <i>cause</i> of leakage and <i>necessary</i> for correct behavior in sparse regions—a fundamental tension.
4	α_{neg}	0.1–2.0	B	Non-negativity penalty. Penalizes negative peak values via $[\max(0, -\hat{x}_i)]^2$. At 0.1 (Exp. 1.2–1.4): mild enforcement, reduces negative fraction from 29.5% to 21.1%. At 2.0 (Exp. 1.5/v7): $20\times$ stronger, yet negative fraction <i>increases</i> to 30.5% because competing gradient directions from 12 loss terms overwhelm it.

#	α name	Range used	Stage	What it controls & why this value
5	α_{baseline}	0.5–2.0	B	Masked baseline reconstruction. The single most impactful loss term. Supervises the baseline in non-peak regions via a ground-truth-derived binary mask. At 1.0 (Exp. 1.3): produces best peak correlation (0.793) and best peak MSE (9.30×10^{-3}). At 2.0 (Exp. 1.5/v7): doubled, but improvements are marginal due to over-regularization. Provides the gradient signal that pure peak reconstruction cannot: “your baseline is wrong here.”
6	α_{smooth}	0.01–0.2	B	Baseline smoothness. Penalizes second derivative (curvature) of $\hat{\mathbf{f}}$. At 0.01: gentle enforcement, allows moderate drift curvature while penalizing sharp leakage bumps. Higher values (0.2) produce very smooth baselines but can under-fit rapid drift changes. Encodes the physical prior that baseline drift is a slowly varying, low-frequency phenomenon.
7	$\alpha_{\text{baseline_tv}}$	0.0–0.1	B	Baseline third derivative. Catches localized bumps that the second-derivative penalty misses (a bump has large third derivative at its edges where curvature changes sign). At 0.1 (Exp. 1.4/v5): active, complements smoothness penalty. At 0.0 (Exp. 1.5/v7): disabled because redundant when envelope constraint is active. Complementary to α_{smooth} —second derivative catches curvature, third derivative catches <i>transitions</i> in curvature.
8	α_{leakage}	0.3–1.0	B	High-frequency baseline leakage. Penalizes high-frequency content in the baseline at peak locations (the observable signature of leakage). Uses the second derivative of $\hat{\mathbf{f}}$ as a proxy for high-frequency content, masked to peak regions. At 0.3 (Exp. 1.4): moderate penalty. At 1.0 (Exp. 1.5): full strength. Despite increasing from 0.3 to 1.0, BLI barely moves ($0.592 \rightarrow 0.611$), confirming that this is a loss-level intervention against an architectural problem.

#	α name	Range used	Stage	What it controls & why this value
9	α_{ortho}	0.1–0.5	B	Peak-baseline orthogonality. Penalizes baseline gradient at peak locations, weighted by ground-truth peak magnitude (detached). At 0.5 (Exp. $1.4/v5$): produces best BLI (0.592) and best baseline MSE (7.40×10^{-3}). The <code>.detach()</code> on ground-truth weights prevents the penalty from vanishing when peaks collapse to zero (which is precisely the leakage scenario). Conceptually motivated by orthogonal NMF—where one component is active, the other should be locally constant.
10	$\alpha_{\text{asym}} = 1.0$	1.0 (fixed)	B	Asymmetric baseline loss (with internal asymmetry ratio $\alpha_{\text{ratio}} = 0.9$). Penalizes baseline over-estimation $9\times$ more than under-estimation. Directly targets leakage: when peak energy leaks into baseline, $\hat{f}_i > f_i$ at peak locations. The 9:1 ratio strongly discourages this, even at the cost of slight under-estimation elsewhere. $\alpha_{\text{ratio}} = 0.9$ was chosen heuristically; sensitivity sweep was not performed.
11	$\alpha_{\text{envelope}} = 0.5$	0.5	C	Envelope constraint. Penalizes baseline exceeding the local minimum of the observed signal, computed via differentiable soft-minimum (log-sum-exp, $\tau = 0.1$, window = 51). Only activated in Stage C (full refinement) because it is computationally expensive and its early activation was found to slow convergence without improving final performance. The soft-min target is detached to prevent the network from gaming the local minimum computation.
12	$\alpha_{\text{freq}} = 0.05$	0.05	C	Frequency separation. Penalizes high-frequency baseline content (above f_c) and low-frequency peak content (below f_c) in the Fourier domain. Very low weight (0.05) because it is a regularizer, not a primary objective. Computationally expensive (requires FFT) and typically disabled during training for efficiency. Enforces the same spectral separation as the Butterworth filter, but via a soft penalty.

Loss Function Specifications

Complete Mathematical Specifications of All 12 Loss Terms

Notation: $\hat{x}_i$ = predicted peak, x_i = true peak, $\hat{f}_i$ = predicted baseline, f_i = true baseline, y_i = observed signal, N = signal length, m_i = binary peak mask (1 at peak locations, 0 elsewhere), f_c = Butterworth cutoff frequency.

Term 1: Reconstruction (Huber)

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^N \ell_{\delta}(\hat{x}_i - x_i), \quad \ell_{\delta}(u) = \begin{cases} \frac{1}{2}u^2 & |u| \leq \delta \\ \delta(|u| - \frac{1}{2}\delta) & |u| > \delta \end{cases}$$

Answer Strategy

Specs: $\delta = 1.0$ (main experiments), $\delta = 0.1$ (progressive experiments, Sec. 4.2). On normalized $[0, 1]$ data with $\delta = 1.0$, almost all residuals fall in the quadratic regime, so Huber $\approx$ MSE. The linear tail provides robustness to large leakage-induced residuals. **Group:** Reconstruction fidelity. **Active:** Stages A, B, C. **Target:** General reconstruction accuracy (anchor term).

Term 2: ℓ_1 Sparsity

$$\mathcal{L}_{\ell_1} = \frac{1}{N} \sum_{i=1}^N |\hat{x}_i|$$

Answer Strategy

Specs: Normalized by signal length N . No smoothing—uses true absolute value (subgradient at zero). **Group:** Sparsity priors. **Active:** Stages B, C. **Target:** False peaks in flat regions. **Side effect:** Causes baseline leakage in dense-peak regions by over-shrinking non-zero values.

Term 3: Total Variation

$$\mathcal{L}_{\text{tv}} = \frac{1}{N-1} \sum_{i=1}^{N-1} |\hat{x}_{i+1} - \hat{x}_i|$$

Answer Strategy

Specs: Anisotropic TV (sum of absolute first differences). Normalized by $N - 1$. **Group:** Sparsity priors. **Active:** Stages B, C. **Target:** Non-sharp peak transitions (promotes piecewise-constant signals). **Side effect:** Like ℓ_1 , also causes shrinkage in dense regions.

Term 4: Non-Negativity

$$\mathcal{L}_{\text{neg}} = \frac{1}{N} \sum_{i=1}^N [\max(0, -\hat{x}_i)]^2$$

Answer Strategy

Specs: Squared hinge loss on negative values. Smooth gradient proportional to violation magnitude (unlike a linear penalty which has constant gradient). Only fires for $\hat{x}_i < 0$. **Group:** Sparsity priors. **Active:** Stages B, C. **Target:** Negative peak artifacts (chromatographic peaks are physically non-negative). Complements the architectural asymmetric soft-thresholding (r^k) in the ISTA layers.

Term 5: Masked Baseline Reconstruction

$$\tau = \max\left(\gamma \cdot \max_i |x_i|, \tau_{\min}\right), \quad m_i = \mathbf{1}[|x_i| \geq \tau]$$
$$\mathcal{L}_{\text{baseline}} = \frac{\sum_{i=1}^N (1 - m_i) \cdot (\hat{f}_i - f_i)^2}{\sum_{i=1}^N (1 - m_i) + \epsilon}$$

Answer Strategy

Specs: $\gamma = 0.02$ (relative threshold), $\tau_{\min} = 10^{-4}$ (absolute floor), $\epsilon = 10^{-8}$ (division safety). Mask is derived from ground-truth $\mathbf{x}$ (used only during training). Denominator normalizes by number of background samples, so loss magnitude is independent of peak density. **Group:** Baseline supervision. **Active:** Stages B, C. **Target:** Baseline error in non-peak regions. **Why critical:** Provides the *only* direct gradient signal about baseline quality. Without it, the optimizer has no way to discourage the baseline from absorbing peak energy.

Term 6: Baseline Smoothness

$$\mathcal{L}_{\text{smooth}} = \frac{1}{N-2} \sum_{i=1}^{N-2} [\hat{f}_{i+2} - 2\hat{f}_{i+1} + \hat{f}_i]^2$$

Answer Strategy

Specs: Squared second finite difference of $\hat{\mathbf{f}}$. Equivalent to $\|D_2 \hat{\mathbf{f}}\|_2^2 / (N-2)$. Quadratic penalty: allows moderate curvature (gentle drift) while strongly penalizing sharp localized features (leakage bumps). **Group:** Baseline supervision. **Active:** Stages B, C. **Target:** High-curvature baseline (encodes physical prior: drift is slowly varying).

Term 7: Baseline Third Derivative

$$\mathcal{L}_{\text{base_tv}} = \frac{1}{N-3} \sum_{i=1}^{N-3} [\hat{f}_{i+3} - 3\hat{f}_{i+2} + 3\hat{f}_{i+1} - \hat{f}_i]^2$$

Answer Strategy

Specs: Squared third finite difference of $\hat{\mathbf{f}}$. A leakage bump has large second derivative at its *center* but large third derivative at its *edges* (curvature sign change). This term catches the edges that Term 6 misses. **Group:** Baseline supervision. **Active:** Stages B, C (disabled at 0.0 in v7). **Target:** Localized baseline bumps.

Term 8: High-Frequency Baseline Leakage

$$\mathcal{L}_{\text{leakage}} = \frac{\sum_{i=1}^{N-2} m'_i \cdot [D_2 \hat{\mathbf{f}}]_i^2}{\sum_{i=1}^{N-2} m'_i + \epsilon}$$

Answer Strategy

Specs: m'_i = peak mask restricted to interior samples (indices $2, \dots, N-1$). D_2 = second-difference operator. Uses second derivative as a proxy for high-frequency content (the Butterworth high-pass $\approx$ second-derivative operator at low frequencies). Normalized per-peak-sample (by $\sum m'_i$), not per-total-sample. **Group:** Leakage-specific. **Active:** Stages B, C. **Target:** Peak energy appearing as high-frequency content in the baseline at peak locations.

Term 9: Peak-Baseline Orthogonality

$$\mathcal{L}_{\text{ortho}} = \frac{1}{N-1} \sum_{i=1}^{N-1} w_i \cdot [\hat{f}_{i+1} - \hat{f}_i]^2, \quad w_i = \frac{|x_i|}{\max_j |x_j| + \epsilon} \quad (. \text{detach}())'d$$

Answer Strategy

Specs: Weights w_i are derived from ground-truth $\mathbf{x}$ and **detached** from the computational graph. If predicted $\hat{\mathbf{x}}$ were used instead, the penalty would vanish when peaks collapse to zero—precisely the leakage scenario. Detached ground truth keeps the penalty active regardless of current prediction quality. **Group:** Leakage-specific. **Active:** Stages B, C. **Target:** Baseline “tracking” peak shapes (non-zero baseline gradient at peak locations).

Term 10: Asymmetric Baseline Loss

$$\mathcal{L}_{\text{asym}} = \frac{1}{N} \sum_{i=1}^N \begin{cases} \alpha \cdot (\hat{f}_i - f_i)^2 & \text{if } \hat{f}_i > f_i \\ (1 - \alpha) \cdot (\hat{f}_i - f_i)^2 & \text{if } \hat{f}_i \leq f_i \end{cases}$$

Answer Strategy

Specs: Internal asymmetry ratio $\alpha = 0.9$ (9:1 over- vs under-estimation penalty). Over-estimation ($\hat{f}_i > f_i$) is the signature of leakage: baseline rises above truth to absorb peak energy. The 9:1 ratio strongly discourages this, at the cost of tolerating mild under-estimation. Inspired by the AsLS/arPLS family’s asymmetric reweighting, but applied in the loss rather than the iterative solver. **Group:** Leakage-specific. **Active:** Stages B, C. **Target:** Baseline over-estimation (the directional signature of leakage).

Term 11: Envelope Constraint

$$\mathcal{L}_{\text{envelope}} = \frac{1}{N} \sum_{i=1}^N \left[\max\left(0, \hat{f}_i - \tilde{y}_i^{\min}\right) \right]^2$$

$$\tilde{y}_i^{\min} = -\tau \log \sum_{j \in \mathcal{W}_i} \exp(-y_j/\tau)$$

Answer Strategy

Specs: Window size $|\mathcal{W}_i| = 51$ samples ($\approx$ half a typical peak width). Temperature $\tau = 0.1$ (tight soft-min approximation). $\hat{y}_i^{\min}$ is **detached** from the computational graph to prevent the network from gaming the local minimum computation. ReLU ensures only *violations* (baseline above local signal minimum) are penalized. **Group:** Leakage-specific. **Active:** Stage C only (expensive; early activation slows convergence). **Target:** Baseline exceeding observed signal (physical impossibility in peak-free regions).

Term 12: Frequency Separation

$$\mathcal{L}_{\text{freq}} = \frac{1}{M} \sum_{m=1}^M \left[|\hat{F}_m|^2 \cdot \mathbf{1}[\nu_m > f_c] + |\hat{X}_m|^2 \cdot \mathbf{1}[\nu_m \leq f_c] \right]$$

Answer Strategy

Specs: $\hat{X}_m, \hat{F}_m$ = DFT coefficients of $\hat{\mathbf{x}}, \hat{\mathbf{f}}$. ν_m = frequency of bin m . $f_c = 0.006$ (Butterworth cutoff). M = number of frequency bins. First term: penalizes high-freq baseline (above f_c). Second term: penalizes low-freq peaks (below f_c). Enforces the same spectral separation as the Butterworth filter, but via soft penalty. **Group:** Leakage-specific. **Active:** Stage C only (requires FFT; typically disabled for efficiency). **Target:** Spectral role violation (peaks should be high-freq, baseline should be low-freq).

100 Basic Questions & Short Answers

Friendly Questions — The Ones That Help You Shine

These are straightforward clarification questions a committee member might ask to understand your work better. They are not adversarial—they are opportunities to demonstrate command of the material. Each answer should be 2–4 sentences.

Chromatography Basics

1. *What is chromatography used for?*

Chromatography separates chemical compounds in a mixture so they can be identified and quantified. It is widely used in pharmaceutical QC, environmental monitoring, food safety, clinical diagnostics, and metabolomics. The detector records a time-series signal whose peaks correspond to individual analytes.

2. *What is a chromatogram?*

A chromatogram is the detector response plotted as a function of time (or elution volume). It consists of peaks—each corresponding to a separated analyte—superimposed on a baseline drift and corrupted by noise. The area under each peak is proportional to the analyte’s concentration.

3. *What is baseline drift and what causes it?*

Baseline drift is a slowly varying, low-frequency distortion that shifts the signal away from its true zero level. It is caused by column bleed (thermal degradation of the stationary phase), detector aging, temperature gradients, and compositional changes in the mobile phase during gradient elution.

4. *Why is baseline correction important?*

Analyte concentration is determined by integrating peak area above the baseline. If the baseline is wrong, the integrated area is wrong, and the reported concentration is inaccurate. In regulated industries, even 2% error can violate acceptance criteria.

5. *What is peak area integration?*

Peak area integration computes the area between the corrected signal and the estimated baseline for each peak. This area is proportional to the mass of analyte present. It is the standard quantification method in chromatography.

6. *What is the signal model you use?*

$\mathbf{y} = \mathbf{x} + \mathbf{f} + \mathbf{w}$, where $\mathbf{y}$ is the observed signal, $\mathbf{x}$ is the peak component, $\mathbf{f}$ is the baseline drift, and $\mathbf{w}$ is additive noise. The goal is to estimate $\mathbf{f}$ from $\mathbf{y}$ so that $\hat{\mathbf{x}} = \mathbf{y} - \hat{\mathbf{f}} - \hat{\mathbf{w}}$ faithfully recovers the peaks.

7. *What types of chromatography does this apply to?*

The method is general—it applies to any chromatographic technique that produces a continuous detector signal: HPLC (liquid chromatography), GC (gas chromatography), ion chromatography, size-exclusion chromatography, etc. The synthetic data was calibrated against LC and GC signals.

8. *What is an EMG peak?*

An exponentially modified Gaussian—a Gaussian convolved with an exponential decay. It is the theoretically justified model for chromatographic peak shapes that accounts for extra-column band broadening. The ratio τ/σ controls the degree of tailing.

9. *What is a bi-Gaussian peak?*

A peak model that uses different Gaussian standard deviations for the left and right halves (σ and $\sigma + \tau$). It captures asymmetric tailing without requiring a numerical convolution, making it simpler to generate

synthetically. We use it in our data generation (Sec. 4.1.1).

10. What is the difference between LC and GC?

In LC (liquid chromatography), the mobile phase is a liquid; in GC (gas chromatography), it is a gas. GC typically produces narrower peaks and has different baseline drift characteristics (column bleed is more pronounced at high temperatures). Both produce signals that fit the $\mathbf{y} = \mathbf{x} + \mathbf{f} + \mathbf{w}$ model.

The BEADS Algorithm

11. What does BEADS stand for?

Baseline Estimation And Denoising using Sparsity. It was proposed by Ning, Selesnick, and Duval in 2014. It jointly estimates the baseline and denoises the signal by exploiting the sparsity of peaks and the smoothness of the baseline.

12. What are the two key priors that BEADS uses?

- (1) The peak component $\mathbf{x}$ is *sparse* in the derivative domain—peaks occupy a small fraction of the signal.
- (2) The baseline $\mathbf{f}$ is *smooth*—it varies slowly relative to the peaks, occupying a low-frequency subspace.

13. What is the role of the high-pass filter H in BEADS?

H is a zero-phase Butterworth high-pass filter that removes the low-frequency baseline from the signal. In the cost function, the data fidelity term $\frac{1}{2}\|H(\mathbf{y} - \mathbf{x})\|^2$ ensures that only the high-frequency residual (peaks + noise) drives the peak estimate.

14. What is the cutoff frequency f_c ?

The normalized cutoff frequency of the Butterworth filter. It defines the boundary between “baseline frequencies” (below f_c) and “peak frequencies” (above f_c). In our model, $f_c = 0.006$, corresponding to a period of about 167 samples.

15. What is the asymmetry ratio r ?

r controls the penalty asymmetry in the sparsity term $\theta(x)$. Positive values (peaks) are penalized with weight 1, negative values with weight r . Since peaks should be positive, $r > 1$ suppresses negative artifacts more aggressively. Default: $r = 6$.

16. What is majorization-minimization (MM)?

An iterative optimization technique that replaces a hard-to-minimize objective with a simpler *majorizer* (upper bound) that is tight at the current estimate. Minimizing the majorizer is guaranteed to decrease the original objective. BEADS uses MM to handle the non-quadratic penalties θ and ϕ .

17. How many iterations does classical BEADS typically need?

About 20–30 iterations to converge. Each iteration involves solving a banded linear system, which is $O(N)$ with banded solvers. The total cost is approximately $O(KN)$ where K is the number of iterations.

18. What are the six BEADS parameters?

λ_0 (sparsity weight), λ_1 (first-derivative smoothness), λ_2 (second-derivative smoothness), r (asymmetry ratio), f_c (filter cutoff frequency), and d (filter order). All must be specified by the user.

19. Why is BEADS hard to tune?

The six parameters interact non-linearly. The optimal values depend on signal amplitude, noise level, peak density, and baseline shape—all of which vary across instruments and experiments. No single configuration works for all signals.

20. What is a banded matrix?

A matrix where non-zero entries are confined to a band of diagonals near the main diagonal. The BEADS

filter matrices A and B are banded with bandwidth d (the filter order). Banded systems can be solved in $O(N)$ time using specialized solvers.

21. What is the penalty function ϕ in BEADS?

$\phi(z) = |z| - \epsilon_1 \log(|z| + \epsilon_1)$, a smoothed approximation to the absolute value. It is differentiable everywhere (unlike $|z|$) while still promoting sparsity. The constant ϵ_1 controls the smoothing at zero.

22. What is the penalty function θ in BEADS?

The asymmetric sparsity penalty: $\theta(x) = x$ for $x > \epsilon_0$, $\theta(x) = -rx$ for $x < -\epsilon_0$, with a smooth quadratic transition for $|x| \leq \epsilon_0$. It promotes sparsity while penalizing negative values r times more heavily.

23. How does BEADS compute the baseline?

After convergence, the baseline is $\hat{\mathbf{f}} = \mathbf{y} - \hat{\mathbf{x}} - H(\mathbf{y} - \hat{\mathbf{x}})$. This extracts the low-frequency component of the residual $\mathbf{y} - \hat{\mathbf{x}}$ using the complementary low-pass filter $L = I - H$.

24. What are arPLS and airPLS?

Asymmetrically reweighted Penalized Least Squares (arPLS, Baek et al. 2015) and adaptive iteratively reweighted PLS (airPLS, Zhang et al. 2010). Both are iterative smoothing methods that estimate the baseline by fitting a smooth curve below the signal, using asymmetric weights to prevent peaks from pulling the baseline up.

25. How does BEADS compare to arPLS?

BEADS jointly estimates peaks and baseline via a cost function with explicit sparsity and smoothness priors, including a frequency-domain filter. arPLS estimates only the baseline using asymmetric smoothing. BEADS has more parameters (6 vs 1–2) but provides a richer decomposition. Both require manual tuning.

Algorithm Unrolling

26. What is algorithm unrolling?

A technique that maps each iteration of an iterative optimization algorithm to one layer of a neural network. The algorithm’s parameters become learnable, and the network is trained end-to-end. It inherits the structure and inductive biases of the original algorithm while gaining adaptivity from data-driven learning.

27. What is LISTA?

Learned ISTA (Gregor & LeCun, 2010)—the founding work on algorithm unrolling. It unrolls the Iterative Shrinkage-Thresholding Algorithm for sparse coding, learning the weight matrices and thresholds end-to-end. It achieves roughly 20× faster convergence than classical ISTA.

28. What is ISTA?

The Iterative Shrinkage-Thresholding Algorithm—a proximal gradient method for minimizing the sum of a smooth function and a non-smooth penalty (typically ℓ_1). Each iteration consists of a gradient step on the smooth part followed by a proximal (soft-thresholding) step for the ℓ_1 part.

29. What is a proximal operator?

The proximal operator of a function g at point $\mathbf{v}$ is $\text{prox}_g(\mathbf{v}) = \arg \min_{\mathbf{x}} \frac{1}{2} \|\mathbf{x} - \mathbf{v}\|^2 + g(\mathbf{x})$. For $g = \lambda \|\cdot\|_1$, the proximal operator is soft-thresholding. It generalizes projection to non-smooth penalties.

30. What is soft-thresholding?

$\mathcal{S}_\lambda(x) = \text{sign}(x) \max(|x| - \lambda, 0)$. It sets values with magnitude below λ to zero and shrinks larger values toward zero by exactly λ . It is the proximal operator of the ℓ_1 norm.

31. What does “untied parameters” mean in unrolling?

In classical algorithms, the same parameters are used at every iteration. In an unrolled network, each layer

gets its own parameters ($\theta^{(k)}$), allowing different layers to specialize. This breaks convergence guarantees but improves performance.

32. *What is the difference between unrolling and a generic neural network?*

An unrolled network has a prescribed architecture derived from an algorithm—each layer performs a specific algorithmic step. A generic network (CNN, U-Net) has no such structure. Unrolled networks have far fewer parameters, stronger inductive biases, and are more interpretable.

33. *What is parameter efficiency in unrolling?*

Because the network structure encodes domain knowledge (the algorithm), only the algorithm’s parameters need to be learned—not arbitrary weight matrices. LBEADS-NET has 100 scalars vs millions for a typical CNN. This means less training data is needed and the model is less prone to overfitting.

34. *What is inductive bias?*

Prior assumptions embedded in the model architecture that constrain what it can learn. LBEADS-NET’s inductive biases come from BEADS: sparsity of peaks, smoothness of baseline, asymmetric penalization, frequency-domain separation. These help the model learn from limited data.

35. *What is ISTA-Net?*

An unrolled ISTA for image compressive sensing (Zhang & Ghanem, 2018). It replaces the hand-crafted sparsifying transform with a learned nonlinear transform using two convolutions and a ReLU. ISTA-Net+ adds skip connections analogous to ResNet.

36. *What is ADMM-Net?*

An unrolled ADMM for compressive sensing MRI (Yang et al., 2016). Each ADMM iteration maps to a network stage with reconstruction, shrinkage, and multiplier update. The shrinkage function is replaced with a learnable piecewise-linear function.

37. *What does “truncation” mean in unrolling?*

Instead of running the algorithm until convergence (unknown number of iterations), we fix K iterations and unroll exactly K layers. This creates a feed-forward network with fixed depth. The learned parameters compensate for the truncation.

38. *Can unrolled networks overfit?*

Yes, but less than generic networks because the algorithmic structure acts as a regularizer. With only 100 parameters, LBEADS-NET is unlikely to memorize training data. The main risk is the parameters adapting to the specific characteristics of the synthetic training distribution.

39. *What is the relationship between unrolling depth and performance?*

Generally, more layers improve performance up to a point, after which diminishing returns or training instability set in. Classical BEADS converges in ~ 20 iterations, so $K = 20$ is a natural starting point. The depth ablation (Table 5.4) is planned but not yet run.

40. *Does LBEADS-NET use weight sharing across layers?*

No—each layer has independent parameters. Weight sharing (using the same parameters for all layers) would reduce the model to a learned-but-uniform ISTA, losing the ability for layers to specialize. The emergent specialization (Sec. 5.5) confirms that untying is beneficial.

LBEADS-NET Architecture

41. *How many learnable parameters does LBEADS-NET have?*

100 scalars: 5 parameters per layer $\times$ 20 layers. The parameters are λ_0^k (sparsity), λ_1^k (first-derivative penalty), λ_2^k (second-derivative penalty), r^k (asymmetry ratio), and η^k (step size).

42. What is the log-space parameterization?

All learnable parameters must be positive (negative regularization weights are meaningless). We store $\log \lambda$ and recover $\lambda = \exp(\log \lambda)$. This ensures positivity via the exponential, allows unconstrained optimization in log-space, and provides smooth gradients.

43. What is the initialization strategy?

All layers start with identical parameters: $\lambda_0 = 0.01$, $\lambda_1 = 0.5$, $\lambda_2 = 0.5$, $r = 6.0$, $\eta = 0.05$. These are conservative—smaller than classical BEADS defaults—to ensure stable refinement from the high-pass initialization.

44. What is the high-pass initialization?

Instead of starting from $\mathbf{x}^0 = \mathbf{0}$, we compute $\mathbf{x}^0 = \mathbf{y} - L(\mathbf{y})$, the high-pass filtered input. This removes the bulk of the baseline and gives the network a much better starting estimate. Without it, small peaks would be permanently zeroed by soft-thresholding in the first layer.

45. How is the baseline computed?

The baseline is not predicted directly by the layers. After the final layer outputs $\hat{\mathbf{x}} = \mathbf{x}^K$, the baseline is computed as $\hat{\mathbf{f}} = L(\mathbf{y} - \hat{\mathbf{x}})$ —the low-pass filtered residual. This guarantees that $\hat{\mathbf{f}}$ is inherently smooth.

46. What does one ISTA layer do?

Two steps: (1) a gradient step that moves the peak estimate toward the data using the high-pass filtered residual and smoothness penalty gradients, scaled by step size η^k ; (2) an asymmetric soft-thresholding step that enforces sparsity and non-negativity.

47. What is asymmetric soft-thresholding?

A modified soft-thresholding where positive values are shrunk by $\lambda_0 \eta$ and negative values are shrunk by $\lambda_0 \eta r$. Since $r > 1$, negative values have a larger dead zone and are more aggressively suppressed. This encourages positive peaks and penalizes negative artifacts.

48. What are the difference operators D_1 and D_2 ?

D_1 computes first differences: $[D_1 \mathbf{x}]_i = x_{i+1} - x_i$. D_2 computes second differences: $[D_2 \mathbf{x}]_i = x_{i+2} - 2x_{i+1} + x_i$. Their transposes $D_1^\top$, $D_2^\top$ are implemented as adjoint accumulation operations. All are computed in $O(N)$ time with no stored matrices.

49. What is the data fidelity gradient?

$\mathbf{g}_{\text{data}} = H(\mathbf{y} - \mathbf{x}^{k-1}) = (\mathbf{y} - \mathbf{x}^{k-1}) - L(\mathbf{y} - \mathbf{x}^{k-1})$. It is the high-pass filtered residual—the unexplained high-frequency content. It drives the peak estimate toward the data while ignoring the low-frequency baseline.

50. What are the smoothness penalty gradients?

$\nabla_{\mathbf{x}} \mathcal{R}_1 = \lambda_1^k D_1^\top (D_1 \mathbf{x}^{k-1} / (|D_1 \mathbf{x}^{k-1}| + \epsilon))$ and similarly for $\nabla_{\mathbf{x}} \mathcal{R}_2$ with D_2 and λ_2^k . These are subgradients of the smoothed ℓ_1 norms of the first and second differences.

51. Why $O(N^2)$ memory instead of $O(N)$?

The filter matrices A , B , L are stored as dense $N \times N$ arrays because the filter construction uses SciPy (which produces dense arrays). Exploiting the banded structure would require custom PyTorch implementations of banded matrix-vector products with autograd support. This is an engineering limitation.

52. What is the effective threshold $\lambda_0^k \eta^k$?

The product of the sparsity weight and the step size. It determines the dead zone of the soft-thresholding operator—values with magnitude below $\lambda_0^k \eta^k$ are set to zero. The network can control this threshold by adjusting either parameter independently.

53. What is intermediate supervision?

Computing the loss at every intermediate layer output $\mathbf{x}^k$ (for $k = 1, \dots, K$), not just the final output. This provides direct gradient signal to each layer, alleviating gradient attenuation across depth. It was explored but not used in the final v5 model.

54. *What is the LBEADS_NET_Fast class?*

The final architecture implementation using ISTA-style layers. “Fast” refers to the simpler per-layer computation (gradient + proximal) compared to the earlier CG-based LBEADS_NET class, which solved a linear system at each layer.

55. *What framework is the model implemented in?*

PyTorch, using float64 precision. The model is a standard `nn.Module` with parameters stored as `nn.ParameterList` entries and filter matrices as registered buffers. Training uses the Adam optimizer.

56. *Why float64 and not float32?*

The Butterworth filter matrices at $f_c = 0.006$ have eigenvalues spanning several orders of magnitude. In float32, the matrix products can produce NaN gradients during backpropagation. Float64 provides sufficient precision but doubles memory usage.

57. *Can the model handle variable-length signals?*

The 100 learned scalar parameters are signal-length-agnostic. However, the filter matrices must be reconstructed for each new N , and with dense storage, $N > 10,000$ becomes memory-prohibitive. With banded implementations, arbitrary N would work.

58. *What is the MPS bypass?*

Apple Silicon GPUs (MPS backend) do not support float64 operations. The training code explicitly bypasses MPS and falls back to CPU. CUDA GPUs support float64 natively.

Training & Data

59. *How is the training data generated?*

Synthetically: peaks are bi-Gaussian with random positions, widths ($\sigma \in [1.5, 25]$), amplitudes (log-uniform $[15, 2500]$), and optional tailing. Baselines are cubic splines, exponential decays, or polynomial+sinusoidal mixtures. Gaussian noise is added with $\sigma_w \in [2, 6]$.

60. *How many training signals are there?*

200 total: 160 training, 40 test, split 80/20 with a fixed seed (42) for reproducibility. All experiments share the same split.

61. *How are signals normalized?*

Each signal is divided by $\max_i |y_i|$, mapping the observed signal to $[-1, 1]$. The same scale factor is applied to the peak and baseline ground truth, preserving the additive decomposition.

62. *What optimizer is used?*

Adam with learning rate 10^{-3} , default momentum parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$), and gradient clipping at norm 1.0. A step-wise schedule halves the learning rate every 50 epochs.

63. *How many epochs of training?*

30 epochs for the progressive experiments (Sec. 4.4), with batch size 16. This gives 10 gradient steps per epoch, 300 total. The curriculum-trained model uses 100 epochs split across 3 stages.

64. *What is the three-stage curriculum?*

Stage A (peak reconstruction): MSE loss only. Stage B (leakage mitigation): add sparsity, baseline supervision, orthogonality, anti-leakage terms. Stage C (full refinement): all terms active. Stages run

sequentially; parameters carry forward. This prevents the competing gradient directions from destabilizing early training.

65. *Why is curriculum training necessary?*

With 12 loss terms active from epoch 1, the competing gradient directions prevent convergence—Experiment 1.5 shows this directly. The curriculum introduces terms gradually, letting the network first learn basic reconstruction (Stage A) before adding structural constraints (Stage B) and refinement (Stage C).

66. *What is the batch size?*

16 signals per batch. With 160 training signals, each epoch has 10 gradient update steps.

67. *What random seed is used?*

42 for both data generation and model initialization. Using the same seed across experiments ensures that differences in results arise solely from the loss configuration, not from data ordering or initialization stochasticity.

68. *How long does training take?*

Not formally reported. On CPU (the primary training environment due to MPS bypass), each epoch takes on the order of minutes for $N = 4096$ with $K = 20$ layers and dense matrix operations. A full 30-epoch run takes roughly 30–60 minutes.

69. *What is the peak clustering mechanism?*

40% of peaks are placed in clusters of 2–4 peaks with Gaussian-distributed positions (spread 15–80 samples). This creates the dense, overlapping peak configurations that stress-test the sparsity assumption and produce baseline leakage.

70. *What baseline types are generated?*

Three types, selected uniformly at random: (1) cubic spline through random knots, (2) exponential decay with a Gaussian hump, (3) low-degree polynomial plus sinusoidal components. Each type mimics a different real-world drift pattern.

71. *What is the baseline-to-peak ratio?*

The baseline amplitude is scaled so that its peak value stands in a log-uniform ratio from 0.01 to 0.5 relative to the peak signal amplitude. This ensures the dataset includes both low-amplitude baselines (easy) and high-amplitude baselines (challenging).

72. *How many peaks per signal?*

Between 20 and 55, drawn uniformly. This range was calibrated against 8 real chromatographic signals that had 29–58 peaks.

73. *Why synthetic data instead of real data?*

Real chromatograms don’t have ground-truth decompositions—you can’t separately measure the baseline, peaks, and noise. Synthetic data provides exact ground truth for all three components, enabling quantitative evaluation.

Evaluation Metrics

74. *What is Peak MSE?*

$\frac{1}{N} \sum_i (\hat{x}_i - x_i)^2$ —the mean squared error between predicted and ground-truth peak signals. Lower is better. Reported as $\times 10^{-3}$ for readability.

75. *What is Peak Correlation?*

The Pearson correlation coefficient between $\hat{\mathbf{x}}$ and $\mathbf{x}$. It measures shape similarity independent of scale—1.0 means perfect shape recovery. Values below ~ 0.95 indicate systematic shape distortion.

76. What is Baseline MSE?

$\frac{1}{N} \sum_i (\hat{f}_i - f_i)^2$ —same as Peak MSE but for the baseline component. Lower is better.

77. What is Area Error?

$|\sum_i \hat{x}_i - \sum_i x_i| / \sum_i x_i \times 100\%$ —the relative error in total integrated peak area. 0% means perfect area preservation. This is the most practically important metric for quantitative chromatography.

78. What is the Baseline Leakage Index (BLI)?

$BLI = \sum_i \max(0, \hat{f}_i - f_i) \cdot m_i / (\sum_i x_i \cdot m_i + \epsilon)$. It measures how much the predicted baseline *over-estimates* the true baseline at peak locations, normalized by peak energy. $BLI = 0$ means no leakage; higher means more peak energy absorbed into baseline.

79. What is PBER?

Peak-to-Baseline Error Ratio: RMSE of the baseline at peak locations divided by RMSE at background locations. $PBER \approx 1.0$ means uniform baseline accuracy; $PBER \gg 1.0$ means the baseline is systematically worse where peaks are—the hallmark of leakage.

80. What is the Negative Fraction?

The fraction of predicted peak samples with $\hat{x}_i < 0$. Should be near zero since chromatographic peaks are physically non-negative. Non-zero values indicate over-correction artifacts.

81. What is the peak mask?

A binary mask $m_i = \mathbf{1}[|x_i| \geq \tau]$ where $\tau = \max(\gamma \cdot \max_j |x_j|, \tau_{\min})$ with $\gamma = 0.02$. It identifies which samples are “at peak locations” vs “at background locations.” Used by BLI, PBER, and the masked baseline loss.

82. Why are BLI and PBER novel metrics?

Prior work on baseline correction reports only global metrics (MSE, correlation, area error). BLI and PBER are the first metrics specifically designed to detect and quantify the *directional* leakage phenomenon—baseline over-estimation localized at peak positions.

83. Why not use SNR or PSNR?

SNR and PSNR measure overall signal quality but don’t distinguish between different types of error. A model could have good PSNR while suffering from severe leakage (the errors cancel when averaged globally). BLI and PBER target the specific failure mode we care about.

Results

84. What is the main quantitative result?

All five LBEADS-NET variants outperform classical BEADS (with default parameters) on peak MSE ($\sim 2\times$ improvement), baseline MSE ($\sim 1.7\times$), and area error ($99.99\% \rightarrow 49\text{--}58\%$). The best configuration is Exp. 1.3 with 6 active loss terms.

85. Which experiment configuration is best?

Experiment 1.3 (+Baseline Supervision) achieves the best peak MSE (9.30×10^{-3}) and best peak correlation (0.793). Experiment 1.4 (+Orthogonality, v5) achieves the best baseline MSE (7.40×10^{-3}) and best BLI (0.592).

86. What is baseline leakage?

The phenomenon where peak energy is erroneously absorbed into the baseline estimate. The estimated baseline rises above the true baseline at peak locations, and the peak estimate is correspondingly suppressed.

Caused by the ℓ_1 shrinkage bias in sparsity-based methods.

87. What is the BLI plateau?

BLI ≈ 0.59 across all LBEADS-NET configurations, varying by only 0.005 from Experiments 1.1 through 1.4. This near-invariance shows that leakage is determined by the architecture (the ℓ_1 -based proximal operator), not the loss function.

88. What is over-regularization?

When too many loss terms create conflicting gradient directions, the optimizer cannot satisfy all objectives and produces a compromise worse than simpler configurations. Experiment 1.5 (12 terms) performs worse than Experiment 1.3 (6 terms) on multiple metrics.

89. What is emergent parameter specialization?

The phenomenon where layers learn different parameter values despite being initialized identically. λ_0 increases monotonically ($0.48 \rightarrow 2.0$), r follows a U-shape ($3.36 \rightarrow 2.0 \rightarrow 9.67$), and η stays roughly constant (~ 0.2). Early layers do coarse separation; later layers do aggressive refinement.

90. How does LBEADS-NET perform on real data?

Qualitative evaluation only (Sec. 5.6). The model produces smooth, plausible baselines on real LC signals. In sparse regions, baseline tracking looks correct. In dense regions, leakage is difficult to assess visually without ground truth.

91. What is the hybrid inference pipeline?

A two-step post-processing procedure: (1) adaptive noise thresholding using the MAD of the high-pass residual, clamped to non-negative; (2) iterated low-pass baseline recomputation (L^p with $p = 3$) for a smoother final baseline. Applied after the network’s raw output.

92. What does the loss term ablation show?

The ablation tables (Tables 5.3, 5.4) are currently empty—experiments have not been run yet. The expected findings are that baseline supervision is critical, non-negativity improves peak quality, and simpler configurations match complex ones given limited training budget.

93. Why does Experiment 1.5 perform worse?

12 competing loss terms produce conflicting gradients. The asymmetric baseline loss pushes the baseline down; the envelope constraint also pushes down; the orthogonality loss can push up. With only 300 gradient steps, the optimizer cannot converge on all 12 objectives simultaneously.

Concepts & Miscellaneous

94. What is the autograd boundary?

The point where PyTorch’s automatic differentiation cannot track gradients—specifically, the SciPy-based filter construction. Operations in SciPy are opaque to PyTorch, so $\partial L / \partial f_c$ cannot be computed. This is why f_c is fixed rather than learned.

95. What is the difference between v5 and v7?

v5 is the thesis model: 8 loss terms, unclamped exponential parameterization, no softplus output. v7 adds 4 anti-leakage terms (12 total), stronger non-negativity ($\alpha_{\text{neg}} = 2.0$), softplus final activation, and clamped parameters. v7 performs *worse* due to over-regularization.

96. What is group sparsity and why is it the top future direction?

Group sparsity ($\ell_{2,1}$ penalty) penalizes *groups* of coefficients jointly: $\sum_g \|\mathbf{x}_g\|_2$. Within an active group, individual amplitudes are NOT shrunk—only entire groups are zeroed or preserved. This eliminates the per-element shrinkage bias that causes leakage.

97. *What is the significance of the negative result?*

The BLI plateau at ~ 0.59 proves that loss engineering cannot overcome an architectural limitation. This redirects future research toward structural solutions (group sparsity, non-convex penalties, hybrid architectures) rather than more loss terms. Negative results save other researchers time.

98. *What would you do differently if you started over?*

Start with group sparsity instead of ℓ_1 , implement banded matrix operations from the beginning, include arPLS as a comparison baseline, and generate a larger and more diverse synthetic dataset. The ISTA reformulation and curriculum training were good decisions worth keeping.